ProtoPaja

Paperinen työpaketti

Kolme Unity-peliprototyyppiä pelimusiikin kurssille, parityönä

Työviikot 1–18 · luovutus työviikon 18 lopussa

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

Aikataulu on muokattava ehdotus. Työviikot ovat parin omasta aloituksesta, eikä paketissa ole kalenteripäivämääriä.

Repositoryyn ei laiteta henkilötietoja, salasanoja eikä muiden ihmisten nimiä. Henkilöistä kirjataan vain rooli.

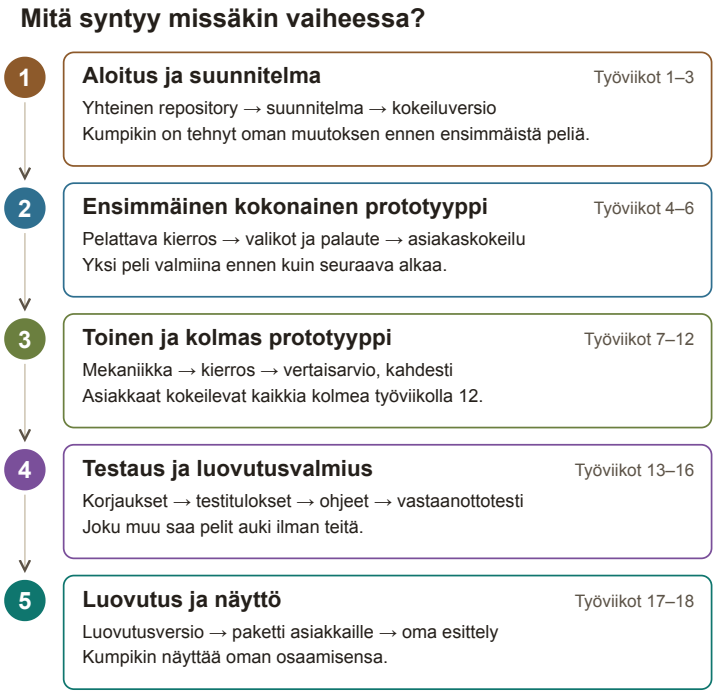
Rasti tässä vihossa tai sivustolla ei ole palautus: työnäyte on aina parin yhteisessä Git-repositoryssa.

Mitä rakennat ja miten etenet

ProtoPajassa teette pelimusiikin kurssille kolme pientä peliä, joihin kurssin opiskelijat tekevät musiikit ja ääniefektit. Jokaisessa pelissä on alkuvalikko, yksi selkeä päämekaniikka, 1–3 minuutin kierros, uudelleenaloitus ja paluu alkuvalikkoon. Pelin pitää olla ymmärrettävä myös ilman ääniä.

1. Tasohyppely: hahmo hyppii esteiden yli, kerää esineet ja pääsee maaliin.
2. Pulmapeli: laatikot siirretään tavoiteruutuihin, ja alkutilanteen voi palauttaa.
3. Toimintapeli: pelaaja selviää areenalla 60 sekuntia yhtä vihollistyyppiä vastaan.
4. Jokaisessa pelissä on selkeä loppu, uudelleenaloitus ja paluu alkuvalikkoon.
5. Äänitapahtumat on kuvattu niin, että musiikkikurssi löytää ne koodista.

Projektin 5 vaihetta



1. Aloitus ja suunnitelma · työviikot 1–3. Saatte saman Unity-projektin toimimaan kummankin koneella, sovitte kolmesta pelistä ja työnjaosta ja kokoatte ensimmäisen kokeiluversion sovitulle alustalle. Tekninen pohja tehdään ensin, koska kahden tekijän ristiriidat ja alustan ongelmat on halvinta ratkaista, kun peliä ei vielä ole.

2. Ensimmäinen kokonainen prototyyppi · työviikot 4–6. Rakennatte ensimmäisestä pelistä kokonaisen kierroksen, alkuvalikon, uudelleenaloituksen ja visuaalisen palautteen. Asiakkaat kokeilevat peliä työviikolla 6, ja heidän palautteensa ohjaa kahta seuraavaa peliä.

3. Toinen ja kolmas prototyyppi · työviikot 7–12. Teette kaksi seuraavaa peliä samalla rytmillä: ensin päämekaniikka, sitten kokonainen kierros ja lopuksi testaus. Toinen peli vertaisarvioidaan ennen kolmannen aloitusta, ja työviikolla 12 asiakkaat kokeilevat kaikkia kolmea.

4. Testaus ja luovutusvalmius · työviikot 13–16. Korjaatte palautteen perusteella valitut asiat, testaatte kaikki kolme peliä koottuina versioina ja kirjoitatte ohjeet jatkotyöstöä varten. Lopuksi joku muu avaa projektit pelkän kirjallisen ohjeen avulla.

5. Luovutus ja näyttö · työviikot 17–18. Luovutatte asiakkaille sovitun paketin: pelattavat versiot, muokattavat Unity-projektit ja ohjeet. Viimeisellä viikolla kumpikin esittelee omat ratkaisunsa ja tekee niihin muutoksen.

Viikon työvaihe on tämän sivuston työohje. Kirjatkaa jokainen työtehtävä tehtävätaulun kortiksi, jossa on vastuuhenkilö ja valmis kun -ehto. Testin tulos kirjataan testimatriisiin heti. Oma viikkoyhteenveto kirjataan projektipäiväkirjaan viikon lopussa ja työaika yhteiseen Excel-taulukkoon joka työkerralla.

Aikataulu

Vko	Ajoitus	Viikon aihe	Vaihe
1	Työviikko 1 / 18	Aloitus: ensimmäinen yhteinen muutos	1
2	Työviikko 2 / 18	Suunnitelma: kolme peliä ja vastuut	1
3	Työviikko 3 / 18	Tekninen pohja ja kokeilukenttä	1
4	Työviikko 4 / 18	Ensimmäinen pelattava kierros	2
5	Työviikko 5 / 18	Valikot, uudelleenaloitus ja palaute	2
6	Työviikko 6 / 18	Ensimmäinen asiakaskokeilu	2
7	Työviikko 7 / 18	Toinen peli: päämekaniikka ja kenttädata	3
8	Työviikko 8 / 18	Toinen peli: kokonainen kierros	3
9	Työviikko 9 / 18	Vertaisarvio ja kolmannen pelin rajaus	3
10	Työviikko 10 / 18	Kolmas peli: päämekaniikka	3
11	Työviikko 11 / 18	Kolmas peli: kokonainen kierros	3
12	Työviikko 12 / 18	Asiakaskatselmointi: kolme peliä	3
13	Työviikko 13 / 18	Palautteen korjaukset	4
14	Työviikko 14 / 18	Testausviikko: koottu peliversio	4
15	Työviikko 15 / 18	Jatkotyöstöohjeet ja äänitapahtumat	4
16	Työviikko 16 / 18	Vastaanottotesti	4
17	Työviikko 17 / 18	Luovutusversio	5
18	Työviikko 18 / 18	Oman osaamisen esittely	5

Palautus viimeistään työviikon 18 lopussa. Sivusto: tehtävien tarkat ohjeet, toteutusavut ja projektipäiväkirja.

Vaihe 1 – Aloitus ja suunnitelma

Työviikko 1 / 18 – Aloitus: ensimmäinen yhteinen muutos

Yhteys kokonaisprojektiin: Ennen ensimmäistä peliä projekti alkaa kysymyksistä ja yhteisestä työympäristöstä. Tällä viikolla varmistatte, että kahden koneen muutokset kulkevat saman repositoryn kautta, koska jokainen myöhempi viikko nojaa siihen. Kysymysten vastauksista tulee työviikolla 2 suunnitelma.

Viikon tavoite: Molemmilla on koneellaan sama Unity-kokeiluprojekti, ja kumpikin on tehnyt siihen oman näkyvän muutoksen, joka on tallessa yhteisessä repositoryssa.

GitHub Desktop: kuvaohje ilman Git-komentoja

Työaikataulukon pohja (.csv, avautuu Excelissä)

Työvaihe 1 / 5 · Kirjoittakaa kysymykset asiakkaille ja ohjaajille

- ☐ Lukekaa toimeksianto. Lukekaa sivun Toimeksianto-osio yhdessä kokonaan. Merkitkää jokaisen prototyypin kahdeksan vaatimusta.
- ☐ Merkitkää epäselvät kohdat. Merkitkää kohdat, joista ette tiedä, mitä asiakkaat tarkoittavat tai milloin asia on valmis.
- ☐ Kirjoittakaa kysymyslista. Kirjoittakaa tiedostoon `kysymykset.md` vähintään kahdeksan kysymystä. Merkitkää jokaisen kysymyksen perään, onko se asiakkaille vai ohjaajille.
- ☐ Tarkistakaa, että kysymys pakottaa päätöksen. Muotoilkaa kysymykset niin, että vastaus muuttaa jotain toteutuksessa. Katsokaa tehtävän esimerkki.
- ☐ Viekää kysymykset vastaajille. Käykää lista läpi ohjaajan kanssa ja lähettäkää asiakkaille tarkoitetut kysymykset sovitulla tavalla. Kirjatkaa vastaukset kysymysten alle.

Valmis kun: Kysymyslistassa on vähintään kahdeksan kysymystä, joista jokaisella on vastaanottaja, ja lista on käyty läpi ohjaajan kanssa.

Tallenna työnäyte: Tiedosto polkuun `project-docs/kysymykset.md`, kun repository on perustettu työvaiheessa 3. Yksi tärkeä kysymys omaan päiväkirjaasi.

Työvaihe 2 / 5 · Asenna ohjaajan päättämä Unity-versio omalle koneellesi

- ☐ Kysy versio ohjaajalta. Kysy ohjaajalta, mikä Unity-versio asennetaan. Versio on ohjaajan päätös, koska pelit luovutetaan musiikkikurssille samalla versiolla. Suositus on LTS-versio eli pitkään tuettu versio.
- ☐ Asenna Unity Hub. Asenna Unity Hub Unityn lataussivulta ja kirjaudu Unity-tunnuksella. Kysy ohjaajalta, mitä tunnusta käytät.
- ☐ Asenna editori. Valitse Unity Hubissa Installs ja Install Editor. Valitse sitten sovittu versio. Jos sitä ei näy listassa, hae se Unityn versioarkistosta.
- ☐ Valitse moduulit. Valitse asennuksen moduuleista koodieditori ja sovitun kohdealustan build-tuki, esimerkiksi Windows Build Support tai Web Build Support. Jos alusta ei ole tiedossa, lisäät moduulin myöhemmin.
- ☐ Asenna GitHub Desktop. Asenna GitHub Desktop viikon kuvaohjeen mukaan ja kirjaudu GitHub-tunnukseesi.
- ☐ Ota versiot talteen. Kirjaa omaan päiväkirjaasi Unityn tarkka versionumero Unity Hubin Installs-näkymästä ja GitHub Desktopin versio kohdasta Help ja About GitHub Desktop.

Valmis kun: Unity Hubin Installs-näkymässä näkyy sovittu versio, ja GitHub Desktop on kirjautunut GitHub-tunnukseesi.

Tallenna työnäyte: Versionumerot omaan päiväkirjaasi.

Työvaihe 3 / 5 · Perustakaa yhteinen repository ja Unity-kokeiluprojekti

- □ Sopikaa luoja. Sopikaa, kumpi luo repositoryn ja kokeiluprojektin. Toinen kloonaa sen tämän työvaiheen lopussa.
- □ Piilottakaa sähköposti. Kumpikin avaa GitHubissa Settings ja Emails ja valitsee Keep my email addresses private ennen ensimmäistä committia. Kysykää ohjaajalta, millä nimellä olette tekijöinä.
- □ Luokaa repository. Luoja luo GitHubissa repositoryn. Näkyvyydeksi valitaan Private, kunnes ohjaaja päättää julkisuudesta. Add .gitignore -kohtaan valitaan Unity.
- □ Luokaa kokeiluprojekti. Luoja kloonaa repositoryn GitHub Desktopilla ja luo Unity Hubissa sovitulla versiolla 2D-projektin nimeltä Kokeilu kloonatun kansion sisään.
- □ Siirtäkää .gitignore. Luoja siirtää repositoryn juuressa olevan .gitignore-tiedoston Kokeilu-kansioon, Assets-kansion viereen. Unityn pohja toimii vain kansiossa, jossa projekti on.
- □ Tarkistakaa ja viekää GitHubiin. Luoja tarkistaa Changes-listasta, ettei siellä ole Library- tai Temp-kansioita, lisää `project-docs/kysymykset.md`, tekee commitin ja painaa Push origin.
- □ Kloonatkaa toiselle koneelle. Toinen kloonaa repositoryn GitHub Desktopilla ja avaa Kokeilu-kansion Unity Hubin Add-toiminnolla. Unity rakentaa Library-kansion itse, mikä voi kestää muutaman minuutin.

Valmis kun: Repository on GitHubissa yksityisenä, siinä ei ole Library- tai Temp-kansioita, ja kokeiluprojekti aukeaa kummankin koneella.

Tallenna työnäyte: Repositoryn osoite ja ensimmäisen commitin linkki kummankin päiväkirjaan.

Työvaihe 4 / 5 · Tee oma näkyvä muutos ja vie se versionhallintaan

- □ Sopikaa vuorot. Tehkää muutokset vuorotellen, koska molemmat muokkaavat samaa sceneä. Ensimmäinen tekee muutoksensa ja painaa Push origin, ja toinen aloittaa vasta sen jälkeen.
- □ Hae parisi muutokset. Paina GitHub Desktopissa Fetch origin ja sitten Pull origin, jos se näkyy. Avaa projekti vasta tämän jälkeen.
- □ Tee muutos. Tee sceneen yksi näkyvä muutos. Valitse esimerkiksi GameObject-valikosta 2D Object, Sprites ja Square ja vaihda neliön väri Inspectorissa.
- □ Kokeile Play-tilassa. Paina Play ja tarkista, että muutos näkyy. Lopeta Play-tila ennen muita muutoksia, koska Play-tilassa tehdyt muutokset häviävät.
- □ Tallenna scene. Tallenna scene pikanäppäimellä Ctrl+S (Mac: Cmd+S). Tallentamaton scene ei näy GitHub Desktopin Changes-listassa.
- □ Luo oma kansio. Luo kansio `project-docs/omat/OMA-KANSIO`, jossa OMA-KANSIO on ohjaajan kanssa sovittu nimesi tai tunnuksesi. Tallenna sinne tyhjä tiedosto `projektipaivakirja.md`.
- □ Tee commit ja push. Kirjoita commit-viestiksi, mitä muutit, esimerkiksi Lisää punainen neliö. Paina Commit to main ja Push origin ja kerro parillesi, että vuoro vaihtuu.

Valmis kun: GitHubin commit-historiassa on oma commitisi omalla nimelläsi, ja muutos näkyy Play-tilassa.

Tallenna työnäyte: Commitin linkki omaan päiväkirjaasi.

Työvaihe 5 / 5 · Perustakaa tehtävätaulu ja työajan seuranta

- □ Valitkaa taulu. Valitkaa tehtävätauluksi Trello tai muu ohjaajan kanssa sovittu taulu. Kutsukaa toinen ja ohjaajat tauluun, jos ohjaajat haluavat seurata sitä.
- □ Tehkää sarakkeet. Tehkää tauluun sarakkeet Tulossa, Työn alla, Testattavana ja Valmis.
- □ Tehkää ensimmäiset kortit. Tehkää kortti jokaisesta tämän viikon työvaiheesta. Kirjoittakaa korttiin tavoite, vastuuhenkilö ja valmis kun -ehto.
- □ Luokaa työaikataulukko. Luokaa yhteinen Excel-taulukko sovittuun paikkaan, esimerkiksi Teamsiin. Viikon pohjassa ovat valmiina sarakkeet päivämäärä, opiskelija, prototyyppi, tehtävä, käytetty aika ja lyhyt tulos.
- □ Kirjatkaa tämän viikon aika. Kirjatkaa kumpikin oma työaikanne tältä viikolta. Kirjaa yksi rivi jokaisesta työkerrasta.
- □ Kertokaa sijainnit. Kirjoittakaa repositoryn `README.md`-tiedostoon, missä tehtävätaulu ja työaikataulukko ovat. Älkää kirjoittako sinne salasanoja.

Valmis kun: Taulussa on kortti jokaisesta tämän viikon työvaiheesta vastuuhenkilöineen, ja työaikataulukossa on kummankin rivit tältä viikolta.

Tallenna työnäyte: Kuvakaappaus taulusta polkuun `project-docs/tyonaytteet/viikko-1/tehtavataulu.png` ja päivitetty `README.md`.

Viikon lopputarkistus: Kumpikin hakee toisen muutoksen omalle koneelleen ja näkee Play-tilassa molemmat muutokset. Repositoryn historiassa on kummankin oma commit omalla nimellä.

Työnäyte Git-repositoryyn ennen rastia: Repositoryn commit-historia, jossa on kummankin oma commit, project-docs/kysymykset.md ja project-docs/tyonaytteet/viikko-1/tehtavataulu.png.

Työviikko 2 / 18 – Suunnitelma: kolme peliä ja vastuut

Yhteys kokonaisprojektiin: Työviikon 1 kysymykset saivat vastauksia, ja nyt ne muuttuvat suunnitelmaksi. Päätätte pelien pienimmät toimivat versiot ja sen, kuka tekee mitäkin, koska kumpikin tarvitsee omat työnäytteensä jokaisesta osaamisvaatimuksesta. Ensimmäisen pelin kortit ohjaavat työtä työviikosta 3 alkaen.

Viikon tavoite: Suunnitelmassa on kolmen pelin mekaniikka, tavoite ja rajaus, vastuutaulukko kertoo kummankin osat jokaisessa pelissä, ja vahvistamattomat asiat on merkitty avoimiksi.

Avaa suunnitelmalomake

Työvaihe 1 / 5 · Kuvatkaa kolmen pelin pienin toimiva versio

- ☐ Valitkaa kolme aihetta. Valitkaa toimeksiannon aiheista kolme tai ehdottakaa vastaavan laajuisia. Aiheet ovat lähtökohta, ja asiakkaat vahvistavat ne työvaiheessa 5.
- ☐ Kuvatkaa päämekaniikka ja tavoite. Kirjoittakaa suunnitelmalomakkeelle jokaisesta pelistä, mitä pelaaja tekee toistuvasti ja mihin hän pyrkii.
- ☐ Kuvatkaa kierroksen kulku. Kirjoittakaa, miten kierros alkaa, miten se päättyy ja miten pelaaja aloittaa uudelleen. Tavoitepituus on 1–3 minuuttia.
- ☐ Kirjoittakaa pienin toimiva versio. Kirjoittakaa jokaiselle pelille pienin toimiva versio yhdellä virkkeellä. Katsokaa tehtävän esimerkki.
- ☐ Kirjoittakaa, mitä jätätte pois. Kirjatkaa lomakkeen Mitä jätämme pois -kenttään vähintään kolme houkuttelevaa lisäystä, joita ette tee.
- ☐ Ladatkaa suunnitelma. Täyttäkää lomake yhdellä koneella, ladatkaa `suunnitelma.md` ja tallentakaa se polkuun `project-docs/suunnitelma.md`. Tehkää commit ja push.

Valmis kun: Suunnitelma kertoo jokaisesta pelistä päämekaniikan, tavoitteen, kierroksen kulun ja pienimmän toimivan version, ja siinä on vähintään kolme pois jätettyä asiaa.

Tallenna työnäyte: `project-docs/suunnitelma.md` repositoryyn.

Työvaihe 2 / 5 · Vertailkaa projektijakoa omilla mittauksilla

- ☐ Mitatkaa kokeiluprojekti. Katsokaa työviikon 1 kokeiluprojektin Library-kansion koko ja mitatkaa, kauanko projektin avaaminen kestää. Kirjatkaa luvut.
- ☐ Kirjoittakaa kriteerit. Luokaa tiedosto `project-docs/paatokset.md` ja kirjoittakaa siihen kriteerit: vastaanottajan avattavuus, ristiriitariski, yhteinen alkuvalikko, levytila ja buildien määrä.
- ☐ Arvioikaa vaihtoehdot. Kirjoittakaa jokaisesta kriteeristä, miten erilliset projektit ja yksi yhteinen projekti eroavat. Käyttäkää mittaamianne lukuja.
- ☐ Tehkää päätös. Valitkaa vaihtoehto ja perustelkaa se kahdella kriteerillä. Kirjatkaa päätös päätöslokiin päivämäärän kanssa ja suunnitelmalomakkeen kenttään.
- ☐ Kertokaa ohjaajalle. Kertokaa päätös ohjaajalle. Jos hän ei vielä vahvista sitä, merkitkää se avoimeksi asiaksi ja jatkakaa valitsemallanne tavalla.

Valmis kun: Päätöslokissa on vertailu omilla mittauksilla, päätös päivämäärineen ja kaksi perustelua.

Tallenna työnäyte: `project-docs/paatokset.md` repositoryyn.

Työvaihe 3 / 5 · Pilkkokaa ensimmäinen peli tehtävätaulun korteiksi

- □ Valitkaa ensimmäinen peli. Valitkaa, mikä kolmesta pelistä tehdään ensin. Valitkaa se, jonka päämekaniikka on teille selvin.
- □ Jakakaa peli toiminnoiksi. Kirjoittakaa ensimmäisen pelin pienin toimiva versio toiminnoiksi, esimerkiksi liike, hyppy, kerättävä esine, este, maali, alkuvalikko ja loppuruutu.
- □ Asettakaa tärkeysjärjestys. Järjestäkää toiminnot niin, että pienimmän toimivan version toiminnot ovat ensin ja lisäykset viimeisinä.
- □ Tehkää kortit. Tehkää jokaisesta toiminnosta tehtävätaulun kortti, jossa on tavoite ja valmis kun -ehto.
- □ Arvioikaa työmäärä. Kirjoittakaa jokaiseen korttiin arvio työtunneista. Arviota verrataan myöhemmin työaikataulukon toteumaan.

Valmis kun: Ensimmäisen pelin jokaisella toiminnolla on kortti, jossa on tavoite, valmis kun -ehto ja tuntiarvio, ja kortit ovat tärkeysjärjestyksessä.

Tallenna työnäyte: Kuvakaappaus taulusta polkuun `project-docs/tyonaytteet/viikko-2/kortit.png`.

Työvaihe 4 / 5 · Tehkää vastuutaulukko, jossa vastuut vaihtuvat pelien välillä

- □ Luokaa vastuutaulukko. Kopioikaa avun pohja tiedostoon `project-docs/vastuutaulukko.md`.
- □ Jakakaa ensimmäinen peli. Kirjoittakaa ensimmäisen pelin sarakkeeseen jokaiselle osa-alueelle tekijän nimi tai sovittu tunnus.
- □ Vaihtakaa vastuut. Täyttäkää toisen pelin sarake niin, että kumpikin saa osa-alueet, joita ei tehnyt ensimmäisessä pelissä.
- □ Tarkistakaa aukot. Tarkistakaa, että jokainen osa-alue on kummallakin vähintään kerran. Kolmannen pelin sarakkeella täytetään jäljelle jääneet aukot.
- □ Tarkistakaa näyttömatriisi. Avatkaa sivun Näyttömatriisi. Tarkistakaa, että kumpikin saa vastuutaulukon osa-alueista oman työnäytteen jokaiseen vaatimukseen.
- □ Näyttäkää ohjaajalle. Käykää taulukko läpi ohjaajan kanssa viikkotarkistuksessa ja korjatkaa se, jos joltakulta puuttuu osa-alue.

Valmis kun: Vastuutaulukossa jokainen osa-alue on kummallakin vähintään kerran, ja ohjaaja on nähnyt taulukon.

Tallenna työnäyte: `project-docs/vastuutaulukko.md` repositoryyn.

Työvaihe 5 / 5 · Pyytäkää vahvistus avoimiin päätöksiin

- □ Listatkaa vahvistettavat. Katsokaa suunnitelmalomakkeen vahvistettavat kohdat: aiheet, kohdealusta, tekoälyn käyttö, repositoryn julkisuus, lisenssi, tutkinnon perusteet ja katselmoinnit.
- □ Viekää aiheet asiakkaille. Lähettäkää asiakkaille kolmen pelin kuvaukset avun viestipohjalla ja kysykää, sopivatko ne musiikkikurssille ja millä alustalla kurssi pelaa.
- □ Viekää loput ohjaajalle. Käykää muut vahvistettavat asiat läpi ohjaajan kanssa viikkotarkistuksessa.
- □ Kirjatkaa vastaukset. Kirjatkaa saadut vastaukset suunnitelmalomakkeelle ja päätöslokiin. Jättäkää vastaamatta jääneet kentät tyhjiksi. Suunnitelma merkitsee ne silloin avoimiksi. Jatkakaa työtä.

Valmis kun: Jokainen vahvistettava asia on suunnitelmassa joko vastattuna tai avoimena asiana, ja asiakkaille on lähetetty aiheet.

Tallenna työnäyte: Päivitetty `project-docs/suunnitelma.md` ja päätöslokin merkintä viestin lähetyspäivästä.

Viikon lopputarkistus: Kumpikin kertoo vuorollaan ilman muistiinpanoja jokaisen pelin pienimmän toimivan version ja omat vastuunsa ensimmäisessä pelissä. Ohjaaja vertaa kerrottua suunnitelmaan ja vastuutaulukkoon, eikä eroja ole.

Työnäyte Git-repositoryyn ennen rastia: project-docs/suunnitelma.md, project-docs/paatokset.md, project-docs/vastuutaulukko.md ja project-docs/tyonaytteet/viikko-2/kortit.png.

Työviikko 3 / 18 – Tekninen pohja ja kokeilukenttä

Yhteys kokonaisprojektiin: Suunnitelma kertoo, mitä teette ja kuka tekee. Nyt rakennatte pohjan, jonka varaan ensimmäinen peli tehdään: sovitun projektirakenteen, kokeilukentän ja koottavan peliversion. Kun build ja ristiriidan ratkaisu toimivat nyt, työviikolla 4 voitte keskittyä pelin kierrokseen.

Viikon tavoite: Sovitulle alustalle koottu kokeiluversio käynnistyy ilman editoria, kumpikin on yhdistänyt toisen muutokset omaan työhönsä, ja ensimmäisen pelin kokeilukentässä voi kokeilla päämekaniikkaa.

Työvaihe 1 / 5 · Perustakaa sovittu projektirakenne

- ☐ Luokaa projekti. Luokaa työviikon 2 päätöksen mukaan ensimmäisen pelin Unity-projekti tai yhteinen projekti repositoryyn sovitulla versiolla. Käyttäkää 2D-pohjaa, esimerkiksi Universal 2D.
- ☐ Lisätkää .gitignore. Kopioikaa Kokeilu-kansion .gitignore jokaisen uuden Unity-projektin kansioon, Assets-kansion viereen.
- ☐ Tarkistakaa versionhallinnan asetukset. Avatkaa Edit-valikosta Project Settings. Tarkistakaa Editor-kohdasta, että Asset Serialization Mode on Force Text, ja Version Control -kohdasta, että Mode on Visible Meta Files.
- ☐ Luokaa kansiot. Luokaa pelin Assets-kansioon sovitut kansiot, esimerkiksi Scenes, Scripts, Prefabs ja Art. Nimetkää ne suunnitelman nimeämissäännön mukaan.
- ☐ Kirjatkaa rakenne README:hen. Kirjoittakaa repositoryn `README.md`-tiedostoon kansiorakenne ja Unity-versio. Tehkää commit ja push.

Valmis kun: Ensimmäisen pelin projekti aukeaa kummankin koneella sovitulla versiolla, Changes-listassa ei ole Library- tai Temp-kansioita, ja README kertoo rakenteen.

Tallenna työnäyte: Rakennecommit ja päivitetty `README.md` repositoryyn.

Työvaihe 2 / 5 · Harjoittele ristiriidan ratkaisua tekstitiedostolla

- ☐ Sopikaa rivi. Sopikaa, että kumpikin muuttaa `README.md`-tiedoston otsikkorivin eri tavalla.
- ☐ Tee oma muutoksesi. Muuta otsikkoriviä ja tee commit, mutta älä vielä paina Push origin.
- ☐ Ensimmäinen pushaa. Ensin toinen teistä painaa Push origin. Sen jälkeen toinen painaa Pull origin, ja GitHub Desktop ilmoittaa ristiriidasta.
- ☐ Ratkaise ristiriita. Avaa tiedosto editorissa. Valitse, kumpi rivi jää, tai yhdistä rivit. Poista Gitin lisäämät kolme merkkiriviä, jotka näet avun koodilaatikosta.
- ☐ Viimeistele yhdistäminen. Palaa GitHub Desktopiin, paina Continue merge ja sitten Push origin.
- ☐ Vaihtakaa vuoroa. Toistakaa harjoitus niin, että toinen teistä ratkaisee ristiriidan.
- ☐ Kirjatkaa oppi. Kirjoittakaa päätöslokiin yhdessä, mitä opitte ja miksi samaa sceneä ei muokata yhtä aikaa.

Valmis kun: Historiassa on yhdistämiscommit, jonka olet tehnyt itse, eikä `README.md` sisällä Gitin merkkirivejä.

Tallenna työnäyte: Oman yhdistämiscommitisi linkki omaan päiväkirjaasi.

Työvaihe 3 / 5 · Rakentakaa ensimmäisen pelin kokeilukenttä

- ☐ Valitkaa scenen omistaja. Sopikaa, kumpi rakentaa kokeilukentän. Toinen ei muokkaa sceneä sillä aikaa. Merkitkää vuoro tehtävätaulun korttiin.
- ☐ Rakentakaa kenttä. Lisätkää sceneen lattia ja muutama este yksinkertaisina muotoina, esimerkiksi Square-spriteinä. Lisätkää niihin Box Collider 2D -komponentti.
- ☐ Lisätkää pelaaja. Lisätkää pelaajaksi yksi muoto ja siihen komponentit Rigidbody 2D ja Box Collider 2D. Tallentakaa scene, tehkää commit ja push.
- ☐ Jakakaa päämekaniikka. Jakakaa päämekaniikka kahteen osaan vastuutaulukon mukaan. Tasohyppelyssä esimerkiksi osa 1 on liike sivuttain ja osa 2 hyppy.
- ☐ Tehkää kortit. Tehkää kummastakin osasta kortti vastuuhenkilöineen. Kirjoittakaa korttiin, mitä pelaaja tekee ja mitä ruudulla tapahtuu, kun osa toimii.

Valmis kun: Kokeilukenttä on repositoryssa, pelaaja putoaa lattialle Play-tilassa, ja tehtävätaulussa on kaksi päämekaniikan korttia eri vastuuhenkilöillä ja odotetuilla tuloksilla.

Tallenna työnäyte: Kenttäcommit repositoryyn ja kuvakaappaus korteista polkuun `project-docs/tyonaytteet/viikko-3/kortit.png`.

Työvaihe 4 / 5 · Toteuta oma osasi päämekaniikasta

- □ Hae parisi muutokset. Paina GitHub Desktopissa Fetch origin ja Pull origin ennen kuin avaat Unityn.
- □ Luo oma skripti. Luo Scripts-kansioon C#-skripti, jonka nimi kertoo, mitä se tekee, esimerkiksi PlayerMovement tai PlayerJump.
- □ Kirjoita toiminto. Kirjoita skriptiin oman osasi toiminto. Avun koodipohjassa on runko, jonka TODO-kohdat täytät itse.
- □ Liitä skripti pelaajaan. Liitä skripti pelaajan GameObjectiin. Jos scene on toisen vuorossa, sovi hetki, jolloin liität skriptin ja tallennat scenen.
- □ Kokeile Play-tilassa. Kokeile osaasi ja vertaa tulosta kortin odotettuun tulokseen. Korjaa, kunnes ne vastaavat toisiaan.
- □ Tee commit ja push. Tee commit, jonka viesti kertoo toiminnon, ja paina Push origin. Siirrä kortti Testattavana-sarakkeeseen.

Valmis kun: Oma osasi toimii Play-tilassa kortin odotetun tuloksen mukaan, ja commit on historiassa omalla nimelläsi.

Tallenna työnäyte: Oman commitisi linkki tehtävätaulun korttiin ja omaan päiväkirjaasi.

Työvaihe 5 / 5 · Kootkaa kokeiluversio sovitulle alustalle

- □ Avatkaa build-asetukset. Avatkaa File-valikosta Build Profiles. Vanhemmissa Unity-versioissa sama ikkuna on nimeltään Build Settings.
- □ Valitkaa alusta. Valitkaa sovittu kohdealusta. Jos alustaa ei ole vahvistettu, käyttäkää omaa käyttöjärjestelmäänne ja kirjatkaa se päätöslokiin.
- □ Lisätkää scene. Lisätkää kokeilukentän scene Scene List -luetteloon.
- □ Kootkaa peli. Painakaa Build ja valitkaa kansioksi Unity-projektin sisälle tehtävä `Builds`-kansio. Builds-kansio ei mene versionhallintaan.
- □ Käynnistäkää build. Käynnistäkää koottu peli ilman editoria ja kokeilkaa molempia päämekaniikan osia.
- □ Kirjatkaa erot. Kirjatkaa päätöslokiin alusta, build-asetukset ja jokainen ero, jonka huomasitte editorin ja buildin välillä.

Valmis kun: Koottu kokeiluversio käynnistyy ilman Unity-editoria, ja molemmat päämekaniikan osat toimivat siinä.

Tallenna työnäyte: Kuvakaappaus käynnissä olevasta buildista polkuun `project-docs/tyonaytteet/viikko-3/build.png` ja alustan tiedot päätöslokiin.

Viikon lopputarkistus: Käynnistäkää koottu kokeiluversio ilman Unity-editoria ja liikkukaa kokeilukentässä. Kumpikin näyttää omalla koneellaan toisen viimeisimmän muutoksen, ja historiassa on kummankin commit.

Työnäyte Git-repositoryyn ennen rastia: Rakennecommit, oma yhdistämiscommitisi, oma päämekaniikkacommitisi ja project-docs/tyonaytteet/viikko-3/build.png.

Vaihe 2 – Ensimmäinen kokonainen prototyyppi

Työviikko 4 / 18 – Ensimmäinen pelattava kierros

Yhteys kokonaisprojektiin: Päämekaniikkaa on kokeiltu pienessä kentässä. Nyt siitä rakennetaan kokonainen kierros. Valikko ja viimeistellyt grafiikat tulevat työviikolla 5; tällä viikolla ratkaistaan, toimiiko itse peli.

Viikon tavoite: Ensimmäistä peliä voi pelata alusta kierroksen päättymiseen asti, ja lopputulos näkyy ruudulla.

Työvaihe 1 / 4 · Sopikaa kierroksen päättymisen kaksi omaa osaa

- ☐ Valitkaa päättymisehto. Valitkaa suunnitelmasta ensimmäisen pelin päättymisehto, esimerkiksi pelaaja koskettaa maalia tai osuu piikkiin.
- ☐ Kirjoittakaa, mitä näkyy. Kirjoittakaa, mitä pelaaja tekee ja mitä ruudulla sen jälkeen näkyy, esimerkiksi teksti Maali! ja pelaajan pysähtyminen.
- ☐ Jakakaa kahteen osaan. Osa 1 on päättymisen tunnistus ja osa 2 loppuviesti ja pelin pysäytys. Osa 1 kuuluu vastuutaulukon kierroksen logiikan tekijälle ja osa 2 käyttöliittymän tekijälle.
- ☐ Sopikaa liitoskohta. Sopikaa, millä metodilla osat kytkeytyvät, esimerkiksi osa 1 kutsuu metodia EndRound, jonka osa 2 kirjoittaa.
- ☐ Tehkää kortit. Tehkää kummastakin osasta kortti, jossa ovat vastuuhenkilö, odotettu tulos ja sovittu metodin nimi.

Valmis kun: Päättymisehto ja kaksi korttia eri vastuuhenkilöille on kirjattu tehtävätauluun, ja liitoskohdan metodin nimi on sovittu.

Tallenna työnäyte: Kuvakaappaus korteista polkuun `project-docs/tyonaytteet/viikko-4/kortit.png`.

Työvaihe 2 / 4 · Toteuta oma osasi kierroksen lopusta

- ☐ Hae parisi muutokset. Paina Fetch origin ja Pull origin ennen kuin avaat Unityn.
- ☐ Kirjoita oma skripti. Kirjoita oman osasi skripti, esimerkiksi GoalTrigger (osa 1) tai RoundManager (osa 2). Käytä avun koodipohjaa ja sovittua metodin nimeä.
- ☐ Kokeile yksin. Kokeile osaasi Play-tilassa. Osa 1: Console-ikkunaan tulostuu viesti, kun pelaaja koskettaa maalia. Osa 2: metodin kutsu näyttää loppuviestin ja pysäyttää pelin.
- ☐ Estä jatkuminen. Varmista, ettei kierros jatku normaalisti päättymisen jälkeen eikä loppu käynnisty kahdesti.
- ☐ Nimeä pelitapahtuma. Anna pelitapahtuman metodille nimi, joka kertoo tapahtuman, esimerkiksi ReachGoal. Musiikkikurssi etsii tapahtumat näillä nimillä.
- ☐ Tee commit ja push. Tee commit vain omista tiedostoistasi ja paina Push origin. Siirrä kortti Testattavana-sarakkeeseen.

Valmis kun: Oma osasi toimii Play-tilassa odotetun tuloksen mukaan, kierros ei jatku päättymisen jälkeen, ja oma commitisi on historiassa.

Tallenna työnäyte: Oman commitisi linkki korttiin ja omaan päiväkirjaasi.

Työvaihe 3 / 4 · Aloittakaa äänitapahtumien kuvaus

- ☐ Luokaa tiedosto. Luokaa tiedosto `docs/aanitapahtumat.md` ja kopioikaa siihen avun taulukkopohja.
- ☐ Kirjatkaa omat tapahtumat. Kumpikin kirjoittaa taulukkoon omat pelitapahtumansa työviikoilta 3 ja 4, esimerkiksi hyppy ja maaliin saapuminen.
- ☐ Kirjatkaa paikka koodissa. Kirjoittakaa jokaiselle tapahtumalle tiedosto ja metodi, jossa tapahtuma syntyy.
- ☐ Ehdottakaa ääntä. Kirjoittakaa ehdotus musiikkikurssille, esimerkiksi lyhyt hyppyääni tai voittofanfaari. Ääniä ette tee itse.
- ☐ Kokeilkaa, löytyykö kohta. Toinen lukee taulukosta tapahtuman, ja toinen avaa koodista oikean kohdan. Korjatkaa rivi, jos kohta ei löydy heti.

Valmis kun: Äänitapahtumataulukossa ovat ensimmäisen pelin tapahtumat tiedostoineen ja metodeineen, ja jokainen tapahtuma löytyi koodista taulukon avulla.

Tallenna työnäyte: `docs/aanitapahtumat.md` repositoryyn.

Työvaihe 4 / 4 · Testatkaa toistenne osat

- ☐ Luokaa testimatriisi. Luokaa tiedosto `project-docs/testimatriisi.md` avun pohjasta.
- ☐ Kirjoittakaa odotetut tulokset. Kirjoittakaa ennen ajoa testitapaukset T01 ja T02: onnistunut kierros ja tilanne, jossa päättymisehto ei vielä täyty. Kirjatkaa kummankin odotettu tulos.
- ☐ Vaihtakaa testaajaa. Kumpikin testaa toisen osan. Testin tekijäksi kirjataan testaaja, ei toteuttaja.
- ☐ Kirjatkaa tulokset. Kirjatkaa havaittu tulos. Tehkää jokaisesta virheestä kortti tehtävätauluun.
- ☐ Korjatkaa ja testatkaa uudelleen. Toteuttaja korjaa oman osansa virheen. Ajakaa sama testitapaus uudelleen ja kirjatkaa tulos.

Valmis kun: Testitapaukset T01 ja T02 ovat testimatriisissa testaajineen ja tuloksineen. Jos virhe jäi korjaamatta, kortti on taulussa keskeneräisenä ja apua on pyydetty.

Tallenna työnäyte: `project-docs/testimatriisi.md`, korjauscommit ja uusintatestin tulos repositoryyn.

Viikon lopputarkistus: Käynnistäkää peli ja pelatkaa kierros sovittuun päättymiseen asti. Loppuviesti vastaa tapahtunutta, eikä peli jatku normaalisti sen jälkeen. Uuden testikierroksen saa tällä viikolla aloittaa editorin Play-tilasta. Historiassa on kummankin commit, ja kumpikin on testannut toisen osan.

Työnäyte Git-repositoryyn ennen rastia: Oma commitisi, docs/aanitapahtumat.md ja project-docs/testimatriisi.md, jossa ovat testitapaukset T01 ja T02 testaajineen.

Työviikko 5 / 18 – Valikot, uudelleenaloitus ja palaute

Yhteys kokonaisprojektiin: Työviikolla 4 peliin tuli kierros, jonka lopun näkee ruudulla. Nyt kierroksen ympärille rakennetaan alkuvalikko, uudelleenaloitus ja palaute, jotta joku muu kuin te voi pelata. Valikosta tehdään prefab, jota käytätte uudelleen kahdessa seuraavassa pelissä.

Viikon tavoite: Pelin voi käynnistää alkuvalikosta, pelata loppuun, aloittaa uudelleen ja palata alkuvalikkoon, ja pelin tapahtumista näkyy selkeä palaute.

Työvaihe 1 / 4 · Sopikaa valikoiden ja palautteen kaksi omaa osaa

- ☐ Kirjoittakaa ohjausohje. Kirjoittakaa pelin ohjausohje enintään neljällä rivillä: mitä näppäimiä käytetään ja mikä on tavoite.
- ☐ Jakakaa kahteen osaan. Osa 1 on alkuvalikko, ohjausohje ja paluu alkuvalikkoon. Osa 2 on uudelleenaloitus ja visuaalinen palaute. Valitkaa tekijät vastuutaulukon käyttöliittymä- ja logiikkarivien mukaan.
- ☐ Valitkaa palautteen kohteet. Listatkaa tapahtumat, joista pelaaja tarvitsee näkyvän palautteen, esimerkiksi kerätty esine, osuma ja jäljellä olevien esineiden määrä.
- ☐ Sopikaa scenejen vuorot. Alkuvalikko on oma scenensä. Osan 1 tekijä muokkaa tällä viikolla valikkosceneä ja osan 2 tekijä pelikenttää.
- ☐ Tehkää kortit. Tehkää kummastakin osasta kortti, jossa ovat vastuuhenkilö, scene ja odotettu tulos.

Valmis kun: Ohjausohje on kirjoitettu, kummallakin on kortti omasta osastaan, ja scenejen vuorot on merkitty kortteihin.

Tallenna työnäyte: Ohjausohje ja palautteen kohteet suunnitelman ensimmäisen pelin kuvaukseen. Kuvakaappaus korteista polkuun `project-docs/tyonaytteet/viikko-5/kortit.png`.

Työvaihe 2 / 4 · Toteuta oma osasi valikoista ja palautteesta

- ☐ Hae parisi muutokset. Paina Fetch origin ja Pull origin ennen kuin avaat Unityn.
- ☐ Rakenna oma osasi. Osa 1: tee alkuvalikkoscene, jossa ovat pelin nimi, Aloita-painike ja ohjausohje. Osa 2: lisää loppuruutuun Uudelleen-painike ja pelikenttään sovittu palaute.
- ☐ Kytke painikkeet. Liitä painikkeiden On Click -kohtaan oman skriptisi metodi, esimerkiksi StartGame, Restart tai BackToMenu.
- ☐ Lisää scenet build-luetteloon. Osa 1: lisää alkuvalikko ja pelikenttä Build Profiles -ikkunan Scene List -luetteloon, alkuvalikko ensimmäiseksi. Osa 2: tarkista, että pelikenttä on luettelossa.
- ☐ Tee prefab. Osa 1: raahaa valikon Canvas Project-ikkunan Prefabs-kansioon. Osa 2: tee loppuruudusta prefab samalla tavalla.
- ☐ Kokeile ja vie. Kokeile osaasi Play-tilassa alkaen alkuvalikosta. Tee commit ja push ja siirrä kortti Testattavana-sarakkeeseen.
- ☐ Lisää äänitapahtumat. Lisää osasi uudet tapahtumat, esimerkiksi painikkeen painallus tai esineen kerääminen, tiedostoon `docs/aanitapahtumat.md`.

Valmis kun: Oma osasi toimii Play-tilassa alkuvalikosta alkaen, oma prefabisi on Prefabs-kansiossa, ja commit on historiassa omalla nimelläsi.

Tallenna työnäyte: Oman commitisi linkki korttiin ja päiväkirjaan.

Työvaihe 3 / 4 · Tuokaa grafiikkapaketti ja tarkistakaa sen käyttöehdot

- ☐ Valitkaa paketti. Vastuutaulukon grafiikkavastaava valitsee yhtenäisen grafiikkapaketin, esimerkiksi Kenneyn CC0-lisenssillä julkaistun paketin.
- ☐ Tarkistakaa käyttöehdot. Toinen lukee paketin käyttöehdot ja tarkistaa, saako grafiikoita käyttää ja jakaa muokattavassa Unity-projektissa.
- ☐ Kirjatkaa luetteloon. Luokaa `docs/assetit-ja-lisenssit.md` ja kirjatkaa paketin nimi, lähde, tekijä, lisenssi ja käyttökohde.
- ☐ Tuokaa grafiikat. Grafiikkavastaava tuo kuvat pelin Art-kansioon ja vaihtaa pelaajan, esteiden ja maalin muodot kuviksi.
- ☐ Tarkistakaa yhtenäisyys. Pelatkaa kierros ja tarkistakaa, että grafiikat ovat samaa tyyliä ja pelaaja erottuu taustasta.

Valmis kun: Pelissä on yhtenäiset grafiikat, ja jokaisella tuodulla paketilla on rivi assettiluettelossa lähteineen ja lisensseineen.

Tallenna työnäyte: `docs/assetit-ja-lisenssit.md` ja grafiikkacommit repositoryyn.

Työvaihe 4 / 4 · Testatkaa toistuvat kierrokset toistenne koneella

- ☐ Hakekaa ja avatkaa. Hakekaa molemmat viimeisimmät muutokset ja avatkaa peli alkuvalikosta.
- ☐ Kirjoittakaa testitapaukset. Kirjoittakaa ennen ajoa testimatriisiin testitapaukset alkuvalikolle, uudelleenaloitukselle, paluulle alkuvalikkoon ja palautteelle odotettuine tuloksineen.
- ☐ Testatkaa toisen osa. Kumpikin testaa toisen osan. Kirjatkaa testaaja ja havaittu tulos.
- ☐ Pelatkaa kolme kierrosta. Pelatkaa kolme kierrosta peräkkäin: Uudelleen kahdesti ja lopuksi Alkuvalikkoon. Kirjatkaa, jääkö peli pysähdyksiin tai näkyykö jotain kahdesti.
- ☐ Korjatkaa ja testatkaa uudelleen. Toteuttaja korjaa oman osansa virheet. Ajakaa samat testitapaukset uudelleen.

Valmis kun: Testimatriisissa on alkuvalikon, uudelleenaloituksen, paluun ja palautteen testitapaukset testaajineen, ja kolme peräkkäistä kierrosta menee läpi ilman virhettä.

Tallenna työnäyte: Päivitetty `project-docs/testimatriisi.md` ja korjauscommitit repositoryyn.

Viikon lopputarkistus: Kulkekaa alkuvalikosta ohjeen kautta kierroksen loppuun, painakaa Uudelleen, pelatkaa kierros loppuun ja palatkaa Alkuvalikkoon. Toistakaa kolme kertaa peräkkäin ilman virhettä. Jokaisesta olennaisesta tapahtumasta näkyy palaute ilman ääntä. Historiassa on kummankin commit, ja kumpikin on testannut toisen osan.

Työnäyte Git-repositoryyn ennen rastia: Oma commitisi, alkuvalikon ja loppuruudun prefabit, docs/assetit-ja-lisenssit.md ja testimatriisin uudet testitapaukset.

Työviikko 6 / 18 – Ensimmäinen asiakaskokeilu

Yhteys kokonaisprojektiin: Ensimmäinen peli on nyt kokonainen: valikko, kierros, loppu ja uudelleenaloitus. Asiakkaat kokeilevat sitä nyt, koska heidän palautteensa ohjaa kahta seuraavaa peliä. Mitä aikaisemmin kuulette, sopiiko peli musiikkikurssille, sitä vähemmän joudutte muuttamaan myöhemmin.

Viikon tavoite: Asiakkaat ovat kokeilleet ensimmäistä peliä, ja palaute, korjaustarpeet ja havaittu soveltuvuus musiikkikurssille on kirjattu.

Työvaihe 1 / 5 · Valmistelkaa kokeiluversio ja esittely asiakkaille

- ☐ Kootkaa build. Kootkaa ensimmäisestä pelistä build sovitulle alustalle ja kokeilkaa se alusta loppuun.
- ☐ Merkitkää kokeiluversio. Merkitkää kokeiltava commit versiomerkinnällä eli tagilla v0.1: napsauttakaa commitia History-näkymässä hiiren kakkospainikkeella, valitkaa Create Tag ja painakaa sitten Push origin.
- ☐ Tarkistakaa roolit. Vastuutaulukon mukaan toinen esittelee ja toinen kirjaa palautteen. Työviikolla 12 roolit vaihtuvat.
- ☐ Kirjoittakaa esittely. Esittelijä kirjoittaa enintään viiden virkkeen esittelyn ilman teknistä sanastoa: mikä peli on, mitä kokeillaan ja mitä halutaan tietää.
- ☐ Kirjoittakaa kysymykset. Kirjoittakaa kolme kysymystä asiakkaille, esimerkiksi mihin pelin hetkiin musiikki ja äänet sopisivat.

Valmis kun: Build toimii, kokeiluversio on merkitty versiomerkinnällä (tag v0.1), ja esittely ja kolme kysymystä ovat tiedostossa `project-docs/asiakaspalaute.md`.

Tallenna työnäyte: `project-docs/asiakaspalaute.md` ja kokeiluversion versiomerkintä (tag v0.1) repositoryyn.

Työvaihe 2 / 5 · Pitäkää asiakaskokeilu ja kirjatkaa palaute asiakkaan sanoin

- ☐ Esitelkää peli. Esittelijä lukee valmistellun esittelyn ja antaa asiakkaan pelata ilman neuvoja.
- ☐ Seuratkaa pelaamista. Kirjaaja merkitsee, missä kohdissa asiakas epäröi, kysyy tai jää jumiin.
- ☐ Kysykää kysymykset. Esittelijä kysyy valmistellut kolme kysymystä.
- ☐ Kirjatkaa sanatarkasti. Kirjaaja kirjoittaa vastaukset asiakkaan sanoin lainausmerkkeihin. Kirjatkaa kokeilun päivä ja osallistujien roolit, ei nimiä.
- ☐ Kirjatkaa oma tulkinta erikseen. Kirjoittakaa heti kokeilun jälkeen erilliseen osioon, mitä palaute tarkoittaa peleille ja soveltuuko peli musiikkikurssille.

Valmis kun: Palautekirjauksessa ovat päivä, roolit, asiakkaan sanat lainauksina ja erillinen tulkintaosio.

Tallenna työnäyte: Päivitetty `project-docs/asiakaspalaute.md` repositoryyn.

Työvaihe 3 / 5 · Muuttakaa palaute korteiksi ja sopikaa järjestys

- ☐ Erotelkaa korjaustarpeet. Käykää palautekirjaus läpi ja alleviivatkaa jokainen kohta, joka vaatii muutoksen.
- ☐ Tehkää kortit. Tehkää jokaisesta korjaustarpeesta kortti, jossa on viittaus palautteeseen ja valmis kun -ehto.
- ☐ Arvioikaa vaikutus. Merkitkää jokaiseen korttiin, koskeeko se vain ensimmäistä peliä vai myös seuraavia pelejä.
- ☐ Sopikaa järjestys. Järjestäkää kortit: ensin kaikkia pelejä koskevat muutokset ja estävät virheet. Muut korjaukset tehdään työviikolla 13.
- ☐ Tarkistakaa ohjaajan kanssa. Käykää järjestys läpi viikkotarkistuksessa ja kirjatkaa päätös päätöslokiin.

Valmis kun: Jokaisesta korjaustarpeesta on kortti viittauksineen, ja järjestys on kirjattu päätöslokiin.

Tallenna työnäyte: Kuvakaappaus korteista polkuun `project-docs/tyonaytteet/viikko-6/palautekortit.png` ja päätöslokin merkintä.

Työvaihe 4 / 5 · Vertaa omia työaika-arvioitasi toteumaan

- ☐ Kokoa omat korttisi. Kokoa ensimmäisen pelin kortit, joissa olit vastuuhenkilö, ja niiden tuntiarviot.
- ☐ Laske toteuma. Laske työaikataulukosta kunkin kortin toteutuneet tunnit.
- ☐ Kirjaa erot. Kirjoita omaan päiväkirjaasi taulukko, jossa ovat kortti, arvio, toteuma ja ero.
- ☐ Selitä suurin ero. Kirjoita yhdellä virkkeellä, miksi suurin ero syntyi.
- ☐ Päätä muutos. Kirjoita, miten arvioit toisen pelin kortit toisin, esimerkiksi niin, että varaat aikaa testaukselle.

Valmis kun: Omassa päiväkirjassasi on vertailutaulukko ja selitys suurimmalle erolle.

Tallenna työnäyte: Vertailu omaan päiväkirjaasi.

Työvaihe 5 / 5 · Kirjaa kolme Unityn rajoitetta, jotka tulivat sinulle vastaan

- ☐ Käy läpi oma työsi. Käy läpi omat commitisi, päiväkirjasi ja testimatriisin merkinnät työviikoilta 1–5.
- ☐ Valitse kolme tilannetta. Valitse kolme tilannetta, joissa Unity rajoitti tai yllätti, esimerkiksi trigger ei toiminut ilman Rigidbody 2D -komponenttia.
- ☐ Kirjaa jokainen. Kirjoita jokaisesta, mitä yritit, mitä Unity teki, miten ratkaisit ja minkä mahdollisuuden tai rajoitteen opit siitä.
- ☐ Liitä todiste. Liitä jokaiseen havaintoon commit-linkki tai kuvakaappaus.

Valmis kun: Omassa kansiossasi on kolme havaintoa, joissa jokaisessa on oma tilanne, ratkaisu ja commit-linkki tai kuvakaappaus.

Tallenna työnäyte: Tiedosto `project-docs/omat/OMA-KANSIO/unity-havainnot.md` repositoryyn.

Viikon lopputarkistus: Lukekaa palautekirjaus ääneen asiakkaalle tai ohjaajalle ja pyytäkää vahvistus, että kirjaus vastaa sanottua. Jokaisesta korjaustarpeesta on kortti tehtävätaulussa.

Työnäyte Git-repositoryyn ennen rastia: Kokeiluversion versiomerkintä (tag v0.1), project-docs/asiakaspalaute.md, palautekortit, oma arvio–toteumavertailusi ja project-docs/omat/OMA-KANSIO/unity-havainnot.md.

Vaihe 3 – Toinen ja kolmas prototyyppi

Työviikko 7 / 18 – Toinen peli: päämekaniikka ja kenttädata

Yhteys kokonaisprojektiin: Ensimmäinen peli on kokonainen, ja asiakkaiden palaute on korteissa. Toinen peli tehdään samalla rytmillä, mutta vastuut vaihtuvat. Lisäksi pelialue luetaan nyt erillisestä datasta, jotta kenttää voi muuttaa ilman koodia. Työviikolla 8 samasta datasta palautetaan alkutilanne.

Viikon tavoite: Toisen pelin päämekaniikka toimii yksinkertaisella pelialueella, ja pelialue luetaan erillisestä datasta.

Työvaihe 1 / 5 · Sopikaa toisen pelin pienin toimiva versio ja kaksi omaa osaa

- ☐ Lukekaa palaute. Lukekaa työviikon 6 asiakaspalaute ja merkitkää kohdat, jotka koskevat myös toista peliä.
- ☐ Tarkentakaa pienin toimiva versio. Tarkentakaa suunnitelman toisen pelin pienin toimiva versio palautteen perusteella ja kirjatkaa muutos päätöslokiin.
- ☐ Perustakaa pelin projekti. Luokaa toiselle pelille Unity-projekti tai kansio työviikon 3 rakenteen mukaan. Muistakaa myös .gitignore.
- ☐ Jakakaa kahteen osaan. Osa 1 on pelialueen luku datasta ja osa 2 päämekaniikka, esimerkiksi laatikon siirto ja estynyt siirto. Valitkaa tekijät vastuutaulukon mukaan: vastuut vaihtuvat ensimmäisestä pelistä.
- ☐ Sopikaa kokeilu. Jos peli ei ole pulmapeli, kirjoittakaa viikon kokeilu omalle pelillenne: mitä pelaaja tekee ja miten datan muutos näkyy.
- ☐ Tehkää kortit. Tehkää osista kortit vastuuhenkilöineen, odotettuine tuloksineen ja tuntiarvioineen.

Valmis kun: Toisen pelin projekti on repositoryssa, pienin toimiva versio on tarkennettu, ja kummallakin on kortti omasta osastaan.

Tallenna työnäyte: Projektin perustamiscommit, päätöslokin merkintä ja kuvakaappaus korteista polkuun `project-docs/tyonaytteet/viikko-7/kortit.png`.

Työvaihe 2 / 5 · Kokeilkaa kenttädataa kahdella tallennustavalla

- ☐ Piirtäkää pieni kenttä. Piirtäkää paperille viisi kertaa viisi ruudun kenttä: seinät, pelaaja, kaksi laatikkoa ja kaksi tavoiteruutua.
- ☐ Tehkää tekstiversio. Kirjoittakaa sama kenttä tekstitiedostoon tai JSON-tiedostoon, esimerkiksi yksi merkki ruutua kohden. Avun pohjassa on esimerkki.
- ☐ Tehkää ScriptableObject-versio. Tehkää sama kenttä ScriptableObjectina avun koodipohjalla ja täyttäkää se Inspectorissa.
- ☐ Muuttakaa yksi ruutu. Siirtäkää molemmissa versioissa yksi laatikko toiseen ruutuun. Kirjatkaa, kumpi oli nopeampi ja kummassa virhe syntyy helpommin.
- ☐ Valitkaa ja perustelkaa. Valitkaa tallennustapa ja kirjatkaa päätöslokiin kokeilun tulos, valinta ja perustelu. Kirjatkaa valinta myös suunnitelmaan.

Valmis kun: Päätöslokissa on sama kenttä kahdella tavalla, kokemus kummankin muuttamisesta ja perusteltu valinta.

Tallenna työnäyte: Molemmat kokeiluversiot ja päätöslokin merkintä repositoryyn.

Työvaihe 3 / 5 · Toteuta oma osasi toisen pelin mekaniikasta

- ☐ Hae parisi muutokset. Paina Fetch origin ja Pull origin ennen kuin avaat Unityn.
- ☐ Kirjoita oma skripti. Osa 1: kirjoita skripti, joka lukee kentän valitusta datasta ja luo ruudut sceneen. Osa 2: kirjoita pelaajan siirto ja laatikon työntäminen. Käytä avun pohjaa.
- ☐ Käsittele virheellinen syöte. Osa 1: jos datassa on tuntematon merkki, tulosta viesti Console-ikkunaan. Peli ei saa kaatua. Osa 2: estä siirto seinän tai toisen laatikon läpi.
- ☐ Kokeile Play-tilassa. Kokeile osaasi ja vertaa tulosta kortin odotettuun tulokseen.
- ☐ Nimeä ja kirjaa tapahtumat. Nimeä pelitapahtumien metodit tapahtuman mukaan, esimerkiksi PushBox ja BlockedMove, ja lisää ne tiedostoon `docs/aanitapahtumat.md`.
- ☐ Tee commit ja push. Tee commit vain omista tiedostoistasi ja paina Push origin. Siirrä kortti Testattavana-sarakkeeseen.

Valmis kun: Oma osasi toimii Play-tilassa kortin odotetun tuloksen mukaan, virheellinen syöte ei kaada peliä, ja oma commitisi on historiassa.

Tallenna työnäyte: Oman commitisi linkki korttiin ja omaan päiväkirjaasi.

Työvaihe 4 / 5 · Tuokaa toisen pelin grafiikat

- ☐ Valitkaa grafiikat. Vastuutaulukon grafiikkavastaava valitsee pelille yhtenäiset grafiikat. Saman paketin käyttö kuin ensimmäisessä pelissä on sallittua.
- ☐ Tarkistakaa käyttöehdot. Toinen tarkistaa, että käyttöehdot sallivat jakamisen muokattavassa projektissa.
- ☐ Kirjatkaa luetteloon. Lisätkää jokaisesta uudesta paketista tai kuvasta rivi tiedostoon `docs/assetit-ja-lisenssit.md`.
- ☐ Tuokaa ja vaihtakaa. Grafiikkavastaava tuo kuvat Art-kansioon ja vaihtaa ruutujen prefabeihin oikeat kuvat.
- ☐ Tarkistakaa erottuvuus. Pelatkaa ja tarkistakaa, että pelaaja, laatikot, seinät ja tavoiteruudut erottuvat toisistaan myös muodon perusteella, ei pelkän värin.

Valmis kun: Toisen pelin grafiikat ovat yhtenäiset, ruututyypit erottuvat muodon perusteella, ja assettiluettelossa on rivi jokaisesta uudesta assetista.

Tallenna työnäyte: Grafiikkacommit ja päivitetty `docs/assetit-ja-lisenssit.md` repositoryyn.

Työvaihe 5 / 5 · Testatkaa toistenne osat

- ☐ Kirjoittakaa testitapaukset. Kirjoittakaa ennen ajoa testitapaukset seuraavilla vapailla tunnuksilla, esimerkiksi T10 siirto tyhjään ruutuun, T11 siirto seinää päin ja T12 virheellinen kenttädata. Kirjatkaa odotetut tulokset.
- ☐ Testatkaa toisen osa. Kumpikin testaa toisen osan ja kirjaa itsensä testin tekijäksi.
- ☐ Muuttakaa dataa. Muuttakaa kenttädatassa yksi ruutu ja kokeilkaa, näkyykö muutos käynnistyksessä.
- ☐ Kirjatkaa tulokset. Kirjatkaa havaitut tulokset ja tehkää jokaisesta virheestä kortti.
- ☐ Korjatkaa ja testatkaa uudelleen. Toteuttaja korjaa oman osansa virheet. Ajakaa samat testitapaukset uudelleen.

Valmis kun: Testimatriisissa ovat toisen pelin testitapaukset testaajineen ja tuloksineen, ja datan muutos näkyi pelissä.

Tallenna työnäyte: Päivitetty `project-docs/testimatriisi.md` ja korjauscommitit repositoryyn.

Viikon lopputarkistus: Esimerkiksi pulmapelissä: siirtäkää laatikko tavoiteruutuun ja yrittäkää siirtää laatikkoa seinää päin. Muuttakaa kenttädatassa yksi ruutu ja käynnistäkää peli: muutos näkyy. Jos teette muuta peliä, sopikaa vastaava kokeilu työvaiheessa 1. Historiassa on kummankin commit, ja kumpikin on testannut toisen osan.

Työnäyte Git-repositoryyn ennen rastia: Oma commitisi, päätöslokin tallennustapavertailu, kenttädatan tiedosto ja testimatriisin uudet testitapaukset.

Työviikko 8 / 18 – Toinen peli: kokonainen kierros

Yhteys kokonaisprojektiin: Työviikolla 7 pelialue syntyi datasta, ja päämekaniikka toimi. Nyt pelistä tulee kokonainen kierros: ratkaisun tunnistus, loppuruutu ja paluu alkutilanteeseen samasta datasta. Ensimmäisen pelin valikkoprefab tuodaan mukaan, joten valikkoa ei rakenneta uudelleen.

Viikon tavoite: Toisen pelin kierros päättyy ratkaisuun, ja alkuvalikko, ohje, uudelleenaloitus ja alkutilanteeseen palautus toimivat.

Työvaihe 1 / 4 · Sopikaa toisen pelin kierroksen kaksi omaa osaa

- ☐ Kirjoittakaa ratkaisun ehto. Kirjoittakaa, milloin pulma on ratkaistu, esimerkiksi jokaisessa tavoiteruudussa on laatikko, ja mitä pelaaja silloin näkee.
- ☐ Jakakaa kahteen osaan. Osa 1 on ratkaisun tunnistus ja loppuruutu. Osa 2 on alkutilanteeseen palautus datasta. Osan 2 tekee se, joka ei lukenut kenttää datasta työviikolla 7.
- ☐ Valitkaa palautuksen näppäin. Sopikaa, millä näppäimellä pelaaja palauttaa alkutilanteen, ja lisätkää se ohjausohjeeseen.
- ☐ Sopikaa scenejen vuorot. Sopikaa, kumpi muokkaa pelikenttää ja kumpi liittää valikon, ja merkitkää vuorot kortteihin.
- ☐ Tehkää kortit. Tehkää kummastakin osasta kortti vastuuhenkilöineen ja odotettuine tuloksineen.

Valmis kun: Ratkaisun ehto ja palautuksen näppäin on kirjattu, ja kummallakin on kortti omasta osastaan.

Tallenna työnäyte: Kuvakaappaus korteista polkuun `project-docs/tyonaytteet/viikko-8/kortit.png`.

Työvaihe 2 / 4 · Toteuta oma osasi toisen pelin kierroksesta

- ☐ Hae parisi muutokset. Paina Fetch origin ja Pull origin ennen kuin avaat Unityn.
- ☐ Kirjoita oma skripti. Osa 1: tarkista jokaisen siirron jälkeen, onko pulma ratkaistu, ja näytä loppuruutu. Osa 2: palauta pelaajan ja laatikoiden paikat, kun sovittua näppäintä painetaan.
- ☐ Käytä samaa dataa. Osa 2: lue alkutilanne samasta kenttädatasta kuin työviikolla 7. Älä kopioi paikkoja koodiin, koska muuten kentän muutos ei näy palautuksessa.
- ☐ Estä tuplaloppu. Osa 1: varmista, ettei loppuruutu aukea kahdesti eikä laatikoita voi siirtää ratkaisun jälkeen.
- ☐ Kirjaa tapahtumat. Lisää uudet pelitapahtumat, esimerkiksi laatikko tavoiteruudussa, ratkaisu ja palautus, tiedostoon `docs/aanitapahtumat.md`.
- ☐ Kokeile ja vie. Kokeile osaasi Play-tilassa, tee commit ja push ja siirrä kortti Testattavana-sarakkeeseen.

Valmis kun: Oma osasi toimii Play-tilassa odotetun tuloksen mukaan, palautus käyttää kenttädataa, ja oma commitisi on historiassa.

Tallenna työnäyte: Oman commitisi linkki korttiin ja omaan päiväkirjaasi.

Työvaihe 3 / 4 · Liittäkää ensimmäisen pelin valikko toiseen peliin

- ☐ Tuokaa prefabit. Tuokaa ensimmäisen pelin valikko- ja loppuruutuprefabit toiseen peliin. Erillisissä projekteissa käyttäkää Assets-valikon Export Package- ja Import Package -toimintoja.
- ☐ Vaihtakaa tekstit. Vaihtakaa prefabin asetuksista pelin nimi ja ohjausohje toisen pelin mukaisiksi. Skriptiä ei tarvitse muuttaa.
- ☐ Kytkekää painikkeet. Tarkistakaa, että Aloita, Uudelleen ja Alkuvalikkoon kutsuvat oikeita metodeja ja että scenet ovat Scene List -luettelossa.
- ☐ Kirjatkaa yhteinen osa. Kirjoittakaa `README.md`-tiedostoon, mistä yhteinen valikko löytyy ja miten se tuodaan uuteen peliin.
- ☐ Kokeilkaa kierros. Kulkekaa alkuvalikosta pelin loppuun ja takaisin alkuvalikkoon.

Valmis kun: Toinen peli käynnistyy alkuvalikosta, jossa ovat toisen pelin nimi ja ohje, painikkeet toimivat, ja README kertoo, miten valikko tuodaan uuteen peliin.

Tallenna työnäyte: Valikkocommit ja päivitetty `README.md` repositoryyn.

Työvaihe 4 / 4 · Testatkaa toistenne osat ja koko kierros

- ☐ Kirjoittakaa testitapaukset. Kirjoittakaa ennen ajoa testitapaukset ratkaisulle, palautukselle kesken kierroksen, uudelleenaloitukselle ja paluulle alkuvalikkoon.
- ☐ Testatkaa toisen osa. Kumpikin testaa toisen osan ja kirjaa itsensä testin tekijäksi.
- ☐ Testatkaa rajatapaus. Palauttakaa alkutilanne heti ratkaisun jälkeen ja kesken siirron. Kirjatkaa, mitä tapahtuu.
- ☐ Pelatkaa kolme kierrosta. Pelatkaa kolme kierrosta peräkkäin ja palatkaa välillä alkuvalikkoon.
- ☐ Korjatkaa ja testatkaa uudelleen. Toteuttaja korjaa oman osansa virheet. Ajakaa samat testitapaukset uudelleen.

Valmis kun: Testimatriisissa ovat toisen pelin kierroksen testitapaukset testaajineen, ja kolme peräkkäistä kierrosta menee läpi.

Tallenna työnäyte: Päivitetty `project-docs/testimatriisi.md` ja korjauscommitit repositoryyn.

Viikon lopputarkistus: Esimerkiksi pulmapelissä: ratkaiskaa pulma, palauttakaa alkutilanne kesken kierroksen, aloittakaa uudelleen ja palatkaa alkuvalikkoon. Jos teette muuta peliä, kokeilkaa vastaavasti kierroksen loppu, uudelleenaloitus ja paluu. Historiassa on kummankin commit, ja kumpikin on testannut toisen osan.

Työnäyte Git-repositoryyn ennen rastia: Oma commitisi, valikkocommit, päivitetty README.md ja testimatriisin kierrostestit.

Työviikko 9 / 18 – Vertaisarvio ja kolmannen pelin rajaus

Yhteys kokonaisprojektiin: Toinen peli on pelattavissa alusta loppuun. Ennen kolmatta peliä kumpikin arvioi toisen toteutuksen ja korjaa oman koodinsa virheen, koska samat virheet toistuisivat muuten kolmannessa pelissä. Kolmannen pelin rajaus vahvistetaan nyt, jotta työviikolla 10 voi aloittaa suoraan.

Viikon tavoite: Toinen peli on vertaisarvioitu ja sen estävät virheet korjattu, asiakkaiden palaute toisesta pelistä on varmistettu, ja kolmannen pelin rajaus on vahvistettu.

Työvaihe 1 / 4 · Vertaisarvioi parisi osat toisesta pelistä

- ☐ Valitse arvioitavat osat. Katso vastuutaulukosta, mitkä toisen pelin osat parisi teki.
- ☐ Pelaa ja kirjaa. Pelaa kolme kierrosta ja kirjaa jokainen poikkeama prototyypin kahdeksasta vaatimuksesta.
- ☐ Lue koodi. Lue parisi skriptit. Kirjaa yksi asia, joka on helppo ymmärtää, ja yksi, jonka nimeäminen tai rakenne on epäselvä.
- ☐ Luokittele havainnot. Merkitse jokainen havainto: estävä virhe, muu virhe tai parannusehdotus.
- ☐ Käy arvio läpi. Käy arvio läpi parisi kanssa. Hän kertoo, onko havainto ymmärretty oikein.

Valmis kun: Vertaisarviossa on vähintään kolme luokiteltua havaintoa sekä yksi myönteinen ja yksi kehitettävä koodihuomio, ja arvio on käyty läpi parin kanssa.

Tallenna työnäyte: Tiedosto `project-docs/omat/OMA-KANSIO/vertaisarvio-peli2.md` ja estävistä virheistä kortit tehtävätauluun.

Työvaihe 2 / 4 · Korjaa oman koodisi virhe virheenkorjausketjuna

- ☐ Valitse virhe. Valitse vertaisarvion tai testimatriisin virhe, joka on omassa koodissasi.
- ☐ Kirjoita toistamisohje. Kirjoita askel askeleelta, miten virhe saadaan esiin, ja kokeile ohjetta kerran.
- ☐ Etsi syy. Etsi syy esimerkiksi Console-ikkunan viestien tai Debug.Log-tulostusten avulla. Kirjaa syy yhdellä virkkeellä.
- ☐ Korjaa. Korjaa virhe ja tee commit, jonka viesti kertoo korjauksen.
- ☐ Testaa uudelleen. Aja toistamisohje uudelleen: virhe ei enää tule esiin.
- ☐ Aja regressiotesti. Pelaa pelin peruskierros alusta loppuun ja tarkista, ettei korjaus rikkonut muuta.

Valmis kun: Testimatriisissa on ketju: havainto, toistamisohje, syy, korjauscommit, uusintatesti ja regressiotesti.

Tallenna työnäyte: Ketju testimatriisiin ja korjauscommitin linkki omaan päiväkirjaasi.

Työvaihe 3 / 4 · Varmistakaa asiakkaiden palaute toisesta pelistä

- ☐ Valmistelkaa kokeiltava. Kootkaa toisesta pelistä build tai lyhyt video, jossa kierros pelataan alusta loppuun.
- ☐ Lähettäkää asiakkaille. Lähettäkää build tai video asiakkaille sovitulla tavalla ja kysykää kaksi kysymystä: sopiiko peli kurssille ja mihin kohtiin ääni sopisi.
- ☐ Kirjatkaa vastaus. Kirjatkaa vastaus asiakkaan sanoin tiedostoon `project-docs/asiakaspalaute.md` päivämäärän kanssa.
- ☐ Kertokaa, jos vastausta ei tule. Jos vastausta ei tule sovittuun päivään mennessä, kertokaa ohjaajalle. Hän päättää, jatketaanko ilman vastausta.

Valmis kun: Asiakaspalautteessa on asiakkaan kommentti toisesta pelistä hänen sanoillaan tai ohjaajan päätös jatkaa ilman sitä.

Tallenna työnäyte: Päivitetty `project-docs/asiakaspalaute.md` repositoryyn.

Työvaihe 4 / 4 · Rajatkaa kolmas peli ja pyytäkää rajaukselle vahvistus

- ☐ Lukekaa suunnitelma. Lukekaa suunnitelman kolmannen pelin kuvaus ja tähänastinen asiakaspalaute.
- ☐ Tarkentakaa pienin toimiva versio. Kirjoittakaa kolmannen pelin pienin toimiva versio ja päättymisehto niin, että ne voi kokeilla.
- ☐ Kirjoittakaa pois jätettävät. Kirjoittakaa kolme asiaa, joita kolmanteen peliin ei tehdä, esimerkiksi toinen vihollistyyppi.
- ☐ Arvioikaa työmäärä. Tehkää kolmannen pelin kortit ja tuntiarviot. Käyttäkää työviikon 6 arvio-toteumavertailun oppia.
- ☐ Pyytäkää vahvistus. Esittäkää rajaus ohjaajalle ja asiakkaille. Kirjatkaa vahvistus tai avoin asia päätöslokiin.

Valmis kun: Kolmannen pelin pienin toimiva versio, päättymisehto ja pois jätettävät asiat on kirjattu, kortit on arvioitu, ja rajaus on vahvistettu tai merkitty avoimeksi.

Tallenna työnäyte: Päivitetty `project-docs/suunnitelma.md` ja päätöslokin merkintä repositoryyn.

Viikon lopputarkistus: Ajakaa vertaisarvion testitapaukset uudelleen korjausten jälkeen. Kaikki estäviksi merkityt virheet on korjattu, ja kummallakin on yksi valmis virheenkorjausketju.

Työnäyte Git-repositoryyn ennen rastia: project-docs/omat/OMA-KANSIO/vertaisarvio-peli2.md, oma virheenkorjausketjusi testimatriisissa, päivitetty project-docs/asiakaspalaute.md ja kolmannen pelin rajaus.

Työviikko 10 / 18 – Kolmas peli: päämekaniikka

Yhteys kokonaisprojektiin: Kolmannen pelin rajaus on vahvistettu, ja kahdesta pelistä on opittu, miten päämekaniikka jaetaan kahteen osaan. Vastuutaulukon viimeinen sarake täyttää aukot: kumpikin tekee osa-alueen, jota ei vielä tehnyt. Työviikolla 11 mekaniikasta tulee kokonainen kierros.

Viikon tavoite: Kolmannen pelin pelaaja liikkuu pelialueella ja käyttää päätoimintoaan, ja vastus reagoi siihen.

Työvaihe 1 / 4 · Sopikaa kolmannen pelin kaksi omaa osaa

- ☐ Perustakaa pelin projekti. Luokaa kolmannelle pelille Unity-projekti tai kansio sovitun rakenteen mukaan ja tuokaa valikkoprefab.
- ☐ Täyttäkää vastuutaulukon aukot. Täyttäkää vastuutaulukon kolmannen pelin sarake niin, että jokainen osa-alue on kummallakin vähintään kerran.
- ☐ Jakakaa kahteen osaan. Esimerkiksi toimintapelissä osa 1 on pelaajan liike ja hyökkäys ja osa 2 vihollinen ja osuma. Osan 2 asetukset, esimerkiksi nopeus ja ilmestymisväli, luetaan datasta.
- ☐ Valitkaa datan tekijä. Asetusten luvun datasta tekee se, joka ei lukenut kenttää datasta toisessa pelissä.
- ☐ Tehkää kortit. Tehkää osista kortit vastuuhenkilöineen, odotettuine tuloksineen ja tuntiarvioineen.

Valmis kun: Kolmannen pelin projekti on repositoryssa, vastuutaulukossa ei ole aukkoja, ja kummallakin on kortti omasta osastaan.

Tallenna työnäyte: Projektin perustamiscommit, päivitetty `project-docs/vastuutaulukko.md` ja kuvakaappaus korteista polkuun `project-docs/tyonaytteet/viikko-10/kortit.png`.

Työvaihe 2 / 4 · Toteuta oma osasi kolmannen pelin mekaniikasta

- ☐ Hae parisi muutokset. Paina Fetch origin ja Pull origin ennen kuin avaat Unityn.
- ☐ Kirjoita oma skripti. Osa 1: kirjoita pelaajan liike ja hyökkäys. Osa 2: kirjoita vihollinen, joka liikkuu pelaajaa kohti ja reagoi osumaan. Käytä avun pohjaa.
- ☐ Lue asetukset datasta. Datan tekijä lukee asetukset ScriptableObjectista tai tiedostosta työviikolla 7 valitulla tavalla. Toinen käyttää samoja asetuksia omassa osassaan, jos niitä tarvitaan.
- ☐ Näytä palaute. Lisää omaan osaasi näkyvä palaute, esimerkiksi välähdys osumassa.
- ☐ Kirjaa tapahtumat. Nimeä ja kirjaa pelitapahtumat, esimerkiksi Attack, Hit ja PlayerDamaged, tiedostoon `docs/aanitapahtumat.md`.
- ☐ Kokeile ja vie. Kokeile osaasi Play-tilassa, tee commit ja push ja siirrä kortti Testattavana-sarakkeeseen.

Valmis kun: Oma osasi toimii Play-tilassa odotetun tuloksen mukaan, asetuksen muutos datassa näkyy pelissä, ja oma commitisi on historiassa.

Tallenna työnäyte: Oman commitisi linkki korttiin ja omaan päiväkirjaasi.

Työvaihe 3 / 4 · Tuokaa kolmannen pelin grafiikat

- ☐ Valitkaa grafiikat. Vastuutaulukon grafiikkavastaava valitsee kolmannelle pelille yhtenäiset grafiikat.
- ☐ Tarkistakaa käyttöehdot. Toinen tarkistaa uusien assettien käyttöehdot.
- ☐ Kirjatkaa luetteloon. Lisätkää uudet assetit tiedostoon `docs/assetit-ja-lisenssit.md`.
- ☐ Tuokaa ja vaihtakaa. Grafiikkavastaava tuo kuvat ja vaihtaa pelaajan, vihollisen ja pelialueen muodot kuviksi.

Valmis kun: Kolmannen pelin grafiikat ovat yhtenäiset, ja assettiluettelossa on rivi jokaisesta uudesta assetista.

Tallenna työnäyte: Grafiikkacommit ja päivitetty `docs/assetit-ja-lisenssit.md` repositoryyn.

Työvaihe 4 / 4 · Testatkaa toistenne osat

- ☐ Kirjoittakaa testitapaukset. Kirjoittakaa ennen ajoa testitapaukset liikkeelle, hyökkäykselle, osumalle ja asetuksen muutokselle odotettuine tuloksineen.
- ☐ Testatkaa toisen osa. Kumpikin testaa toisen osan ja kirjaa itsensä testin tekijäksi.
- ☐ Testatkaa rajatapaus. Kokeilkaa esimerkiksi monta vihollista yhtä aikaa tai hyökkäystä seinän läpi ja kirjatkaa tulos.
- ☐ Korjatkaa ja testatkaa uudelleen. Toteuttaja korjaa oman osansa virheet. Ajakaa samat testitapaukset uudelleen.

Valmis kun: Testimatriisissa ovat kolmannen pelin mekaniikan testitapaukset testaajineen ja tuloksineen.

Tallenna työnäyte: Päivitetty `project-docs/testimatriisi.md` ja korjauscommitit repositoryyn.

Viikon lopputarkistus: Esimerkiksi toimintapelissä: liikkukaa areenalla, hyökätkää viholliseen ja katsokaa, että osuma näkyy, ja antakaa vihollisen osua pelaajaan. Muuttakaa datassa yhtä asetusta, esimerkiksi vihollisen nopeutta, ja katsokaa, että muutos näkyy. Jos teette muuta peliä, sopikaa vastaava kokeilu työvaiheessa 1. Historiassa on kummankin commit, ja kumpikin on testannut toisen osan.

Työnäyte Git-repositoryyn ennen rastia: Oma commitisi, päivitetty project-docs/vastuutaulukko.md, asetustiedosto ja testimatriisin uudet testitapaukset.

Työviikko 11 / 18 – Kolmas peli: kokonainen kierros

Yhteys kokonaisprojektiin: Työviikolla 10 kolmannen pelin mekaniikka toimi. Nyt siitä tulee kierros, jolla on kesto ja loppu, ja valikko liitetään samalla tavalla kuin toisessa pelissä. Tämän viikon jälkeen kaikki kolme peliä ovat asiakkaiden kokeiltavissa työviikolla 12.

Viikon tavoite: Kolmannen pelin kierros päättyy selviytymiseen tai häviöön, ja alkuvalikko, ohje, uudelleenaloitus ja palaute toimivat.

Työvaihe 1 / 3 · Sopikaa kolmannen pelin kierroksen kaksi omaa osaa

- ☐ Kirjoittakaa voitto ja häviö. Kirjoittakaa, milloin kierros päättyy voittoon ja milloin häviöön, esimerkiksi selviytyminen 60 sekuntia tai pelaajan kestävyyden loppuminen.
- ☐ Jakakaa kahteen osaan. Osa 1 on ajastin ja kierroksen päättyminen. Osa 2 on valikko, ohjausohje, uudelleenaloitus ja visuaalinen palaute. Valitkaa tekijät vastuutaulukon mukaan.
- ☐ Valitkaa näkyvä aika. Sopikaa, miten jäljellä oleva aika ja pelaajan kestävyys näkyvät ruudulla.
- ☐ Tehkää kortit. Tehkää kummastakin osasta kortti vastuuhenkilöineen ja odotettuine tuloksineen.

Valmis kun: Voiton ja häviön ehdot on kirjattu, ja kummallakin on kortti omasta osastaan.

Tallenna työnäyte: Kuvakaappaus korteista polkuun `project-docs/tyonaytteet/viikko-11/kortit.png`.

Työvaihe 2 / 3 · Toteuta oma osasi kolmannen pelin kierroksesta

- ☐ Hae parisi muutokset. Paina Fetch origin ja Pull origin ennen kuin avaat Unityn.
- ☐ Kirjoita oma skripti. Osa 1: kirjoita ajastin, joka päättää kierroksen voittoon, ja häviö, kun pelaajan kestävyys loppuu. Osa 2: liitä valikko, ohje, uudelleenaloitus ja palaute.
- ☐ Varmista yksi loppu. Osa 1: varmista, että kierros päättyy vain kerran, vaikka voitto ja häviö osuisivat samaan hetkeen. Osa 2: varmista, että uudelleenaloitus nollaa ajan ja kestävyyden.
- ☐ Kirjaa tapahtumat. Kirjaa uudet pelitapahtumat, esimerkiksi viimeiset 10 sekuntia, voitto ja häviö, tiedostoon `docs/aanitapahtumat.md`.
- ☐ Kokeile ja vie. Kokeile osaasi Play-tilassa, tee commit ja push ja siirrä kortti Testattavana-sarakkeeseen.

Valmis kun: Oma osasi toimii Play-tilassa odotetun tuloksen mukaan, kierros päättyy vain kerran, ja oma commitisi on historiassa.

Tallenna työnäyte: Oman commitisi linkki korttiin ja omaan päiväkirjaasi.

Työvaihe 3 / 3 · Testatkaa toistenne osat ja koko kierros

- ☐ Kirjoittakaa testitapaukset. Kirjoittakaa ennen ajoa testitapaukset voitolle, häviölle, uudelleenaloitukselle ja paluulle alkuvalikkoon.
- ☐ Testatkaa toisen osa. Kumpikin testaa toisen osan ja kirjaa itsensä testin tekijäksi.
- ☐ Testatkaa rajatapaus. Kokeilkaa, mitä tapahtuu, jos pelaaja häviää viimeisellä sekunnilla, ja kirjatkaa tulos.
- ☐ Pelatkaa kolme kierrosta. Pelatkaa kolme kierrosta peräkkäin ja tarkistakaa, että aika ja kestävyys alkavat joka kerta alusta.
- ☐ Korjatkaa ja testatkaa uudelleen. Toteuttaja korjaa oman osansa virheet. Ajakaa samat testitapaukset uudelleen.

Valmis kun: Testimatriisissa ovat kolmannen pelin kierroksen testitapaukset testaajineen, ja kolme peräkkäistä kierrosta menee läpi.

Tallenna työnäyte: Päivitetty `project-docs/testimatriisi.md` ja korjauscommitit repositoryyn.

Viikon lopputarkistus: Esimerkiksi toimintapelissä: pelatkaa yksi kierros loppuun, esimerkiksi 60 sekunnin selviytyminen, ja yksi häviöön. Aloittakaa uudelleen ja palatkaa alkuvalikkoon. Jos teette muuta peliä, kokeilkaa vastaavasti molemmat loput. Historiassa on kummankin commit, ja kumpikin on testannut toisen osan.

Työnäyte Git-repositoryyn ennen rastia: Oma commitisi, valikon liitos ja testimatriisin voitto-, häviö- ja uudelleenaloitustestit.

Työviikko 12 / 18 – Asiakaskatselmointi: kolme peliä

Yhteys kokonaisprojektiin: Kaikki kolme peliä ovat kokonaisia kierroksia. Asiakkaat kokeilevat niitä nyt yhdessä, koska vasta kokonaisuudesta näkee, sopivatko pelit kurssin harjoituksiksi. Katselmoinnissa sovitut korjaukset tehdään työviikolla 13.

Viikon tavoite: Asiakkaat ovat kokeilleet kaikkia kolmea peliä, ja tärkeimmät korjaukset ja niiden järjestys on sovittu.

Työvaihe 1 / 4 · Valmistelkaa kolmen pelin katselmointiversio

- ☐ Kootkaa buildit. Kootkaa kaikista kolmesta pelistä build sovitulle alustalle ja kokeilkaa jokainen alusta loppuun.
- ☐ Merkitkää katselmointiversio. Merkitkää katselmoitava commit versiomerkinnällä eli tagilla v0.2 ja painakaa Push origin.
- ☐ Vaihtakaa roolit. Työviikon 6 kirjaaja esittelee nyt, ja esittelijä kirjaa. Tarkistakaa roolit vastuutaulukosta.
- ☐ Kirjoittakaa esittely. Esittelijä kirjoittaa lyhyen esittelyn kolmesta pelistä ilman teknistä sanastoa ja kertoo, mitä työviikon 6 palautteen perusteella muutettiin.
- ☐ Kirjoittakaa kysymykset. Kirjoittakaa kolme kysymystä, esimerkiksi mikä peleistä on kurssille tärkein ja mitä pitää korjata ennen luovutusta.

Valmis kun: Kaikki kolme buildia toimivat, katselmointiversio on merkitty (tag v0.2), ja esittely ja kysymykset ovat asiakaspalautteen tiedostossa.

Tallenna työnäyte: Päivitetty `project-docs/asiakaspalaute.md` ja katselmointiversion versiomerkintä (tag v0.2) repositoryyn.

Työvaihe 2 / 4 · Pitäkää katselmointi ja kirjatkaa palaute

- ☐ Esitelkää pelit. Esittelijä lukee esittelyn ja antaa asiakkaiden pelata jokaista peliä ilman neuvoja.
- ☐ Seuratkaa pelaamista. Kirjaaja merkitsee, missä kohdissa asiakkaat epäröivät tai jäävät jumiin.
- ☐ Kysykää kysymykset. Esittelijä kysyy valmistellut kolme kysymystä.
- ☐ Kirjatkaa sanatarkasti. Kirjaaja kirjoittaa vastaukset asiakkaiden sanoin lainausmerkkeihin pelikohtaisesti. Kirjatkaa päivä ja roolit.
- ☐ Kirjatkaa tulkinta erikseen. Kirjoittakaa heti katselmoinnin jälkeen oma tulkintanne erilliseen osioon.

Valmis kun: Palautekirjauksessa ovat päivä, roolit, pelikohtaiset lainaukset ja erillinen tulkintaosio.

Tallenna työnäyte: Päivitetty `project-docs/asiakaspalaute.md` repositoryyn.

Työvaihe 3 / 4 · Sopikaa korjausten järjestys asiakkaiden kanssa

- ☐ Listatkaa korjaustarpeet. Listatkaa katselmoinnin korjaustarpeet ja aiemmat avoimet korjauskortit.
- ☐ Sopikaa järjestys asiakkaiden kanssa. Kysykää asiakkailta, mitkä korjaukset ovat kurssin kannalta tärkeimmät, ja järjestäkää kortit sen mukaan.
- ☐ Rajatkaa työviikkoon 13. Valitkaa korjaukset, jotka mahtuvat työviikkoon 13. Muut kirjataan tiedossa oleviksi puutteiksi.
- ☐ Kertokaa lisätavoitteista. Kertokaa asiakkaille, että neljännestä ja viidennestä pelistä päätetään ohjaajan kanssa vasta, kun kolme peliä on testattu työviikolla 14.
- ☐ Pyytäkää vastaanottotestaaja. Pyytäkää ohjaajaa nimeämään työviikon 16 vastaanottotestiin projektin ulkopuolinen henkilö.
- ☐ Kirjatkaa päätökset. Kirjatkaa korjausten järjestys, lisätavoitteiden päätösaika ja vastaanottotestaajan rooli päätöslokiin.

Valmis kun: Korjauskortit ovat sovitussa järjestyksessä vastuuhenkilöineen, ja päätöslokissa ovat lisätavoitteiden päätösaika ja vastaanottotestaajan rooli.

Tallenna työnäyte: Päätöslokin merkintä ja kuvakaappaus korteista polkuun `project-docs/tyonaytteet/viikko-12/korjauskortit.png`.

Työvaihe 4 / 4 · Tarkista omat työnäytteesi näyttömatriisista

- ☐ Avaa näyttömatriisi. Avaa sivun Näyttömatriisi ja käy läpi omien tutkinnon osiesi vaatimukset.
- ☐ Merkitse syntyneet. Merkitse rasti niihin vaatimuksiin, joista sinulla on jo oma työnäyte repositoryssa.
- ☐ Listaa aukot. Kirjoita tiedostoon `project-docs/omat/OMA-KANSIO/aukot.md` vaatimukset, joista oma työnäyte puuttuu.
- ☐ Sovi paikkaus. Sovi parisi ja ohjaajan kanssa jokaiselle aukolle tehtävä työviikoilta 13–17, jossa työnäyte syntyy.
- ☐ Arvioi omaa toimintaasi. Kirjoita samaan tiedostoon kolme virkettä omasta toiminnastasi tiimissä: mikä on sujunut ja mitä muutat.

Valmis kun: Aukkolistassa jokaisella puuttuvalla vaatimuksella on sovittu tehtävä, ja itsearvioinnissa on kolme virkettä.

Tallenna työnäyte: `project-docs/omat/OMA-KANSIO/aukot.md` repositoryyn.

Viikon lopputarkistus: Lukekaa katselmoinnin päätökset ääneen asiakkaille tai ohjaajalle ja pyytäkää vahvistus. Jokaisella sovitulla korjauksella on kortti ja vastuuhenkilö, ja kumpikin on listannut omat aukkonsa näyttömatriisista.

Työnäyte Git-repositoryyn ennen rastia: Katselmointiversion versiomerkintä (tag v0.2), päivitetty project-docs/asiakaspalaute.md, päätöslokin merkintä ja project-docs/omat/OMA-KANSIO/aukot.md.

Vaihe 4 – Testaus ja luovutusvalmius

Työviikko 13 / 18 – Palautteen korjaukset

Yhteys kokonaisprojektiin: Työviikolla 12 asiakkaat valitsivat tärkeimmät korjaukset. Nyt ne tehdään niin, että jokaisella on odotettu tulos ennen korjausta ja uusintatesti sen jälkeen. Korjattu versio on työviikon 14 testauksen pohja.

Viikon tavoite: Palautteen perusteella valitut korjaukset toimivat, ja ne on testattu uudelleen.

Työvaihe 1 / 3 · Kirjatkaa valitut korjaukset odotettuine tuloksineen

- ☐ Avatkaa korjauskortit. Avatkaa työviikolla 12 valitut korjauskortit tehtävätaulusta.
- ☐ Kirjoittakaa odotettu tulos. Kirjoittakaa jokaiseen korttiin, mitä pelaaja näkee, kun korjaus on valmis.
- ☐ Lisätkää testitapaukset. Lisätkää jokaisesta korjauksesta testimatriisiin testitapaus, jonka odotettu tulos on kirjattu ennen korjausta.
- ☐ Jakakaa vastuut. Jakakaa kortit niin, että kumpikin korjaa omaa koodiaan ja kummallakin on vähintään yksi korjaus.
- ☐ Sopikaa scenejen vuorot. Merkitkää kortteihin, kuka muokkaa mitäkin sceneä ja milloin.

Valmis kun: Jokaisella valitulla korjauksella on kortti, odotettu tulos, testitapaus ja vastuuhenkilö, ja kummallakin on vähintään yksi korjaus.

Tallenna työnäyte: Päivitetty `project-docs/testimatriisi.md` ja kuvakaappaus korteista polkuun `project-docs/tyonaytteet/viikko-13/kortit.png`.

Työvaihe 2 / 3 · Korjaa omat korjauskorttisi

- ☐ Hae parisi muutokset. Paina Fetch origin ja Pull origin ennen kuin avaat Unityn.
- ☐ Toista ongelma. Toista korjattava ongelma ennen kuin muutat mitään, ja kirjaa toistamisohje korttiin.
- ☐ Etsi syy. Etsi syy ja kirjaa se korttiin yhdellä virkkeellä.
- ☐ Korjaa ja tee commit. Korjaa yksi kortti kerrallaan ja tee jokaisesta korjauksesta oma commit.
- ☐ Aja testitapaus. Aja kortin testitapaus ja kirjaa tulos testimatriisiin.
- ☐ Kirjaa ketju. Kirjaa vähintään yhdestä korjauksesta täydellinen virheenkorjausketju testimatriisiin.

Valmis kun: Omat korjauskorttisi ovat Valmis-sarakkeessa, jokaisella on commit ja läpäisty testitapaus, ja yhdestä on täydellinen virheenkorjausketju.

Tallenna työnäyte: Korjauscommitit ja ketju testimatriisiin, linkit omaan päiväkirjaasi.

Työvaihe 3 / 3 · Testatkaa korjaukset ja peruskierrokset uudelleen

- ☐ Hakekaa uusin versio. Hakekaa molempien korjaukset ja avatkaa jokainen peli.
- ☐ Testatkaa toisen korjaukset. Kumpikin ajaa toisen tekemien korjausten testitapaukset ja kirjaa itsensä testin tekijäksi.
- ☐ Ajakaa regressiotestit. Pelatkaa jokaisen pelin peruskierros alusta loppuun, uudelleenaloitus ja paluu alkuvalikkoon.
- ☐ Kirjatkaa tulokset. Kirjatkaa tulokset testimatriisiin. Tehkää jokaisesta uudesta virheestä kortti.
- ☐ Kirjatkaa puutteet. Kirjatkaa korjaamatta jääneet asiat tiedostoon `docs/testitulokset-ja-puutteet.md` tiedossa olevina puutteina.

Valmis kun: Jokainen korjaus on uusintatestattu toisen tekemänä, jokaisen pelin peruskierros menee läpi, ja korjaamatta jääneet asiat on kirjattu puutteiksi.

Tallenna työnäyte: Päivitetty testimatriisi ja `docs/testitulokset-ja-puutteet.md` repositoryyn.

Viikon lopputarkistus: Ajakaa jokaisen korjauksen testitapaus ja jokaisen pelin peruskierros uudelleen. Kaikki menevät läpi, ja kummallakin on toinen valmis virheenkorjausketju.

Työnäyte Git-repositoryyn ennen rastia: Omat korjauscommitisi, toinen virheenkorjausketjusi ja testimatriisin uusintatestit.

Työviikko 14 / 18 – Testausviikko: koottu peliversio

Yhteys kokonaisprojektiin: Korjaukset on tehty ja testattu editorissa. Nyt testaatte pelit niin kuin musiikkikurssi niitä käyttää: koottuina versioina, useita kierroksia peräkkäin ja rajatapauksissa. Testitulokset kuuluvat luovutukseen, ja ne ratkaisevat, voiko lisätavoitteista keskustella.

Viikon tavoite: Kaikkien kolmen pelin testitulokset on kirjattu, ja aloitus, päättyminen ja toistuvat uudelleenaloitukset toimivat myös kootussa peliversiossa.

Työvaihe 1 / 4 · Täydentäkää testimatriisi kaikista kolmesta pelistä

- ☐ Tarkistakaa kattavuus. Tarkistakaa, että jokaisesta pelistä on testitapaus käynnistykselle, ohjausohjeelle, päämekaniikalle, visuaaliselle palautteelle, päättymiselle, uudelleenaloitukselle ja paluulle alkuvalikkoon.
- ☐ Kirjoittakaa puuttuvat. Kirjoittakaa puuttuvat testitapaukset odotettuine tuloksineen ennen ajoa.
- ☐ Jakakaa testaajat. Jakakaa testaus niin, että kumpikin testaa peliä, jota ei pääosin toteuttanut itse.
- ☐ Ajakaa ja kirjatkaa. Ajakaa testitapaukset editorissa ja kirjatkaa testaaja, versio ja tulos.

Valmis kun: Testimatriisissa on jokaisesta pelistä vähintään seitsemän testitapausta tuloksineen: käynnistys, ohjausohje, päämekaniikka, palaute, päättyminen, uudelleenaloitus ja paluu alkuvalikkoon.

Tallenna työnäyte: Päivitetty `project-docs/testimatriisi.md` repositoryyn.

Työvaihe 2 / 4 · Kootkaa peliversiot ja testatkaa ne ilman editoria

- ☐ Jakakaa pelit. Sopikaa, kuka kokoaa minkäkin pelin. Kumpikin kokoaa ja testaa vähintään yhden.
- ☐ Kootkaa build. Kootkaa build sovitulle alustalle Builds-kansioon.
- ☐ Sulkekaa Unity. Sulkekaa Unity-editori kokonaan ennen testausta, jotta testaatte vain koottua versiota.
- ☐ Ajakaa testitapaukset buildissa. Ajakaa pelin testitapaukset buildissa ja kirjatkaa versioksi build ja commit.
- ☐ Kirjatkaa erot. Kirjatkaa jokainen ero editorin ja buildin välillä virheeksi tai puutteeksi.

Valmis kun: Jokaisen pelin build on testattu ilman editoria, ja tulokset ovat testimatriisissa.

Tallenna työnäyte: Päivitetty testimatriisi ja kuvakaappaus jokaisen pelin buildista kansioon `project-docs/tyonaytteet/viikko-14/`.

Työvaihe 3 / 4 · Testatkaa rajatapaukset ja toistuvat uudelleenaloitukset

- ☐ Kirjoittakaa rajatapaukset. Kirjoittakaa jokaiselle pelille kaksi rajatapausta, esimerkiksi Uudelleen-painikkeen tuplapainallus tai näppäimen painaminen samalla hetkellä, kun kierros päättyy.
- ☐ Pelatkaa kymmenen kierrosta. Pelatkaa jokaisen pelin buildissa kymmenen kierrosta peräkkäin ja tarkistakaa, ettei mikään kasva, hidastu tai jää näkyviin.
- ☐ Kokeilkaa ikkunan kokoa. Kokeilkaa buildia eri ikkunakoossa tai näyttötarkkuudella, jos alusta sen sallii.
- ☐ Kirjatkaa tulokset. Kirjatkaa rajatapausten tulokset testimatriisiin ja tehkää virheistä kortit.

Valmis kun: Jokaisella pelillä on kaksi rajatapausta tuloksineen, ja kymmenen peräkkäistä kierrosta on pelattu jokaisessa buildissa.

Tallenna työnäyte: Päivitetty `project-docs/testimatriisi.md` repositoryyn.

Työvaihe 4 / 4 · Arvioi oman alueesi tietoturva luovutusta varten

- ☐ Listaa luovutettavat tiedostot. Avaa GitHub Desktopin Repository-valikosta komentorivi (Open in Command Prompt tai Open in Terminal) ja aja avun komento. Tallenna tuloste.
- ☐ Tarkista oma alueesi. Käy listasta läpi omat tiedostosi. Merkitse, onko niissä nimiä, sähköpostiosoitteita, salasanoja tai muuta, mikä ei kuulu luovutukseen.
- ☐ Tarkista ulkoiset paketit. Avaa Window-valikosta Package Manager ja kirjaa, mitä paketteja omassa pelissäsi on ja mistä ne tulevat.
- ☐ Testaa virheellinen data. Kirjoita testitapaus, jossa pelin data on virheellistä, esimerkiksi kenttätiedostossa on tuntematon merkki. Aja testi ja kirjaa tulos.
- ☐ Kirjaa arvio. Kirjoita tiedostoon `project-docs/omat/OMA-KANSIO/tietoturva-arvio.md` havainnot, riskit ja korjaukset.

Valmis kun: Tietoturva-arviossa ovat luovutettavien tiedostojen tarkistus, pakettien alkuperä ja virheellisen datan testitapaus tuloksineen.

Tallenna työnäyte: `project-docs/omat/OMA-KANSIO/tietoturva-arvio.md` ja testitapaus testimatriisiin repositoryyn.

Viikon lopputarkistus: Käynnistäkää jokaisen pelin koottu versio ilman editoria. Pelatkaa kolme kierrosta peräkkäin ja palatkaa alkuvalikkoon joka kerta. Testimatriisissa on jokaisesta pelistä vähintään seitsemän testitapausta tuloksineen.

Työnäyte Git-repositoryyn ennen rastia: Päivitetty project-docs/testimatriisi.md, buildien kuvakaappaukset kansiossa project-docs/tyonaytteet/viikko-14/ ja project-docs/omat/OMA-KANSIO/tietoturva-arvio.md.

Työviikko 15 / 18 – Jatkotyöstöohjeet ja äänitapahtumat

Yhteys kokonaisprojektiin: Pelit on testattu koottuina versioina. Nyt kirjoitatte ohjeet, joiden avulla musiikkikurssin opiskelija avaa projektin ja löytää kohdat, joihin äänet liitetään. Ohjeet testataan työviikolla 16 projektin ulkopuolisella henkilöllä.

Viikon tavoite: Jatkotyöstöohjeet, äänitapahtumien kuvaus ja assettiluettelo ovat valmiita kokeiltaviksi, ja tarpeeton aineisto on poistettu luovutuksesta.

Työvaihe 1 / 4 · Kirjoita oma osuutesi avaamis- ja jatkotyöstöohjeesta

- ☐ Sopikaa osuudet. Sopikaa parisi kanssa, kumpi kirjoittaa minkäkin pelin ohjeen. Kummallakin on vähintään yksi peli.
- ☐ Kirjoita avaaminen. Kirjoita oman pelisi avaamisohje: Unity-versio, mikä kansio avataan Unity Hubissa ja mikä scene käynnistetään.
- ☐ Kirjoita rakenne. Kirjoita, missä pelin scenet, skriptit, prefabit ja data ovat ja mitä tärkeimmät skriptit tekevät.
- ☐ Kirjoita pelaaminen. Kirjoita ohjausohje ja kierroksen kulku pelaajan näkökulmasta.
- ☐ Kirjoita jatkotyöstö. Kirjoita, miten kenttää tai asetuksia muutetaan ja mistä äänitapahtumat löytyvät.
- ☐ Kokeile ohjetta. Pyydä parisi avaamaan peli pelkän ohjeesi avulla ja korjaa kohdat, joissa hän epäröi.

Valmis kun: Ohjeessa on oma osuutesi, ja parisi on avannut pelin sen avulla kysymättä.

Tallenna työnäyte: `docs/avaaminen-ja-jatkotyosto.md` repositoryyn ja oman osuutesi commit-linkki päiväkirjaan.

Työvaihe 2 / 4 · Viimeistelkää äänitapahtumien kuvaus

- ☐ Tarkistakaa kattavuus. Pelatkaa jokainen peli ja kirjatkaa jokainen hetki, johon ääni tai musiikki sopisi. Verratkaa taulukkoon.
- ☐ Täydentäkää puuttuvat. Lisätkää puuttuvat tapahtumat tiedostoineen ja metodeineen.
- ☐ Tarkistakaa nimet. Tarkistakaa, että taulukon metodien nimet vastaavat koodia. Jos nimi on epäselvä, muuttakaa nimeä koodissa ja taulukossa.
- ☐ Kirjoittakaa ohje. Kirjoittakaa taulukon alkuun kaksi virkettä siitä, miten musiikkikurssi voi liittää äänen tapahtumaan, esimerkiksi lisäämällä AudioSource-komponentin. Ääniä ette tee itse.
- ☐ Kokeilkaa, löytyvätkö tapahtumat. Toinen valitsee kolme tapahtumaa, ja toinen etsii ne koodista pelkän taulukon avulla. Vaihtakaa vuoroa.

Valmis kun: Taulukossa ovat kaikkien kolmen pelin tapahtumat, nimet vastaavat koodia, ja kolme valittua tapahtumaa löytyi koodista taulukon avulla.

Tallenna työnäyte: Valmis `docs/aanitapahtumat.md` repositoryyn.

Työvaihe 3 / 4 · Viimeistelkää assettiluettelo ja poistakaa tarpeeton aineisto

- ☐ Käykää Art-kansiot läpi. Käykää jokaisen pelin Art-kansio läpi ja tarkistakaa, että jokaisella assetilla on rivi assettiluettelossa.
- ☐ Täydentäkää käyttöehdot. Täydentäkää puuttuvat lähteet ja lisenssit. Jos lähdettä ei löydy, korvatkaa asset luvallisella.
- ☐ Poistakaa käyttämättömät. Poistakaa assetit ja skriptit, joita mikään peli ei käytä, sekä Kokeilu-projekti, jos ohjaaja ei halua säilyttää sitä.
- ☐ Kokeilkaa poiston jälkeen. Avatkaa jokainen peli ja pelatkaa kierros, jotta näette, ettei poisto rikkonut mitään.

Valmis kun: Jokaisella peleissä käytetyllä assetilla on lähde ja lisenssi luettelossa, käyttämätön aineisto on poistettu, ja pelit toimivat poiston jälkeen.

Tallenna työnäyte: Valmis `docs/assetit-ja-lisenssit.md` ja siivouscommit repositoryyn.

Työvaihe 4 / 4 · Siisti oma koodisi jatkotyöstöä varten

- ☐ Listaa omat skriptisi. Listaa skriptit, jotka olet kirjoittanut itse.
- ☐ Tarkista nimet. Tarkista, että muuttujien, metodien ja luokkien nimet kertovat, mitä ne tekevät. Muuta epäselvät nimet.
- ☐ Poista kuollut koodi. Poista kommentoidut kokeilut ja käyttämättömät muuttujat.
- ☐ Lisää alkukommentit. Lisää jokaisen skriptin alkuun kommenttirivi siitä, mitä skripti tekee ja mihin GameObjectiin se liitetään.
- ☐ Aja regressiotesti. Pelaa pelin peruskierros alusta loppuun. Jos muutit metodin nimeä, päivitä se myös äänitapahtumataulukkoon.
- ☐ Tee commit ja push. Tee commit, jonka viesti kertoo siistimisen, ja paina Push origin.

Valmis kun: Omissa skripteissäsi on kuvaavat nimet ja alkukommentit, kuollut koodi on poistettu, ja pelin peruskierros menee läpi.

Tallenna työnäyte: Siistimiscommit ja yksi ennen ja jälkeen -esimerkki omaan päiväkirjaasi.

Viikon lopputarkistus: Toinen teistä valitsee äänitapahtumataulukosta kolme tapahtumaa, ja toinen löytää kunkin toteutuskohdan koodista pelkän taulukon avulla. Vaihtakaa sitten vuoroa. Jokainen kohta löytyy alle minuutissa.

Työnäyte Git-repositoryyn ennen rastia: docs/avaaminen-ja-jatkotyosto.md, docs/aanitapahtumat.md, docs/assetit-ja-lisenssit.md ja oma siistimiscommitisi.

Työviikko 16 / 18 – Vastaanottotesti

Yhteys kokonaisprojektiin: Ohjeet on kirjoitettu ja koodi siistitty. Nyt sisältö jäädytetään luovutusehdokkaaksi, ja projektin ulkopuolinen henkilö kokeilee, saako hän pelit auki ilman teitä. Hänen epäröintinsä kertovat, mitä ohjeeseen pitää vielä korjata ennen luovutusta.

Viikon tavoite: Projektin ulkopuolinen henkilö on avannut Unity-projektit ja käynnistänyt pelit kirjallisen ohjeen avulla, ja havainnot ja korjaukset on kirjattu.

Työvaihe 1 / 3 · Merkitkää luovutusehdokas ja valmistelkaa vastaanottotesti

- ☐ Jäädyttäkää sisältö. Sopikaa, ettei peleihin lisätä tämän jälkeen uutta. Korjataan vain estävät virheet ja ohjeet.
- ☐ Merkitkää luovutusehdokas. Merkitkää jäädytetty commit versiomerkinnällä eli tagilla v0.9 ja painakaa Push origin.
- ☐ Tehkää puhdas klooni. Kloonatkaa repository uuteen kansioon tai toiselle koneelle. Testaaja aloittaa tästä kopiosta, ei teidän työkansiostanne.
- ☐ Kirjoittakaa testaajan tehtävä. Kirjoittakaa testaajalle tehtävä: avaa jokainen peli ohjeen mukaan, pelaa kierros ja etsi kaksi äänitapahtumaa koodista taulukon avulla.
- ☐ Sopikaa aika ja roolit. Sopikaa ohjaajan nimeämän testaajan kanssa aika. Sopikaa, kumpi seuraa ja kumpi kirjaa.

Valmis kun: Luovutusehdokas on merkitty (tag v0.9), puhdas klooni on valmiina, ja testaajan tehtävä on kirjoitettu.

Tallenna työnäyte: Tehtävä tiedostoon `project-docs/vastaanottotesti.md` ja luovutusehdokkaan versiomerkintä (tag v0.9) repositoryyn.

Työvaihe 2 / 3 · Pitäkää vastaanottotesti ja kirjatkaa havainnot

- ☐ Antakaa ohje ja tehtävä. Antakaa testaajalle kirjallinen ohje ja tehtävä. Älkää neuvoko suullisesti.
- ☐ Seuratkaa hiljaa. Kirjatkaa jokainen kohta, jossa testaaja epäröi, kysyy tai tekee jotain muuta kuin ohje tarkoittaa.
- ☐ Kysykää lopuksi. Kysykää lopuksi, mikä oli vaikeinta ja mitä ohjeesta puuttui.
- ☐ Kirjatkaa sanatarkasti. Kirjatkaa testaajan vastaukset hänen sanoinaan lainausmerkkeihin. Kirjatkaa myös päivä ja testaajan rooli.
- ☐ Luokitelkaa havainnot. Merkitkää jokainen havainto: ohjeen korjaus, estävä virhe tai tiedossa oleva puute.

Valmis kun: Vastaanottotestin kirjauksessa ovat päivä, testaajan rooli, epäröintikohdat, testaajan sanat ja luokitellut havainnot.

Tallenna työnäyte: Päivitetty `project-docs/vastaanottotesti.md` repositoryyn.

Työvaihe 3 / 3 · Korjaa omalle alueellesi osuneet havainnot

- ☐ Valitse omat havainnot. Valitse vastaanottotestin havainnoista ne, jotka koskevat omaa peliäsi tai ohjeen osuuttasi.
- ☐ Korjaa ohje. Korjaa ohjeen epäselvät kohdat niin, ettei testaajan kysymystä enää synny.
- ☐ Korjaa estävä virhe. Jos havainto on estävä virhe omassa koodissasi, korjaa se virheenkorjausketjuna ja aja peruskierros uudelleen.
- ☐ Kirjaa puutteet. Kirjaa muut havainnot tiedostoon `docs/testitulokset-ja-puutteet.md` tiedossa olevina puutteina.
- ☐ Tee commit ja push. Tee commit ja push ja merkitse havainnot käsitellyiksi vastaanottotestin tiedostoon.

Valmis kun: Jokainen omalle alueellesi osunut havainto on korjattu ohjeeseen, korjattu estävänä virheenä tai kirjattu tiedossa olevaksi puutteeksi.

Tallenna työnäyte: Korjauscommitit ja päivitetty `docs/testitulokset-ja-puutteet.md` repositoryyn, linkit omaan päiväkirjaasi.

Viikon lopputarkistus: Testaaja avaa projektit ja käynnistää pelit pelkän kirjallisen ohjeen avulla ilman suullista apua. Jokainen kohta, jossa hän epäröi, on korjattu ohjeeseen tai kirjattu tiedossa olevaksi puutteeksi.

Työnäyte Git-repositoryyn ennen rastia: Luovutusehdokkaan versiomerkintä (tag v0.9), project-docs/vastaanottotesti.md, korjattu ohje ja docs/testitulokset-ja-puutteet.md.

Vaihe 5 – Luovutus ja näyttö

Työviikko 17 / 18 – Luovutusversio

Yhteys kokonaisprojektiin: Luovutusehdokas on testattu ulkopuolisella henkilöllä, ja ohje on korjattu. Nyt luovutusversio kootaan ja tarkistetaan puhtaasta kloonista, ja paketti luovutetaan asiakkaille sovitulla tavalla. Luovutuksen jälkeen jäljellä on oman osaamisen esittely.

Viikon tavoite: Sovitut korjaukset on tehty, ohjeet vastaavat toteutusta, ja asiakkaat ovat saaneet sovitun luovutuspaketin.

Työvaihe 1 / 4 · Ajakaa regressiotesti puhtaasta kloonista

- ☐ Tarkistakaa jäljellä olevat. Tarkistakaa, onko tehtävätaulussa estäviä virheitä. Muita muutoksia ei enää tehdä.
- ☐ Korjatkaa estävät. Korjatkaa estävät virheet. Jokainen korjaus saa oman commitin ja testitapauksen.
- ☐ Kloonatkaa puhtaasti. Kloonatkaa repository uuteen kansioon ja avatkaa jokainen peli sieltä.
- ☐ Ajakaa regressiotesti. Pelatkaa jokaisen pelin peruskierros, uudelleenaloitus ja paluu alkuvalikkoon puhtaasta kloonista.

Valmis kun: Estäviä virheitä ei ole, ja jokaisen pelin peruskierros menee läpi puhtaasta kloonista.

Tallenna työnäyte: Regressiotestin tulokset `project-docs/testimatriisi.md`-tiedostoon.

Työvaihe 2 / 4 · Kokoa ja tarkista oman pelisi luovutusbuild

- ☐ Valitse pelisi. Sovi parisi kanssa, mitkä pelit kumpikin kokoaa. Kummallakin on vähintään yksi.
- ☐ Tarkista asetukset. Tarkista Build Profiles -ikkunasta alusta, scenet ja niiden järjestys: alkuvalikko ensimmäisenä.
- ☐ Kokoa puhtaasta kloonista. Kokoa build puhtaasta kloonista, ei omasta työkansiostasi.
- ☐ Testaa build. Käynnistä build ilman editoria ja pelaa kierros, uudelleenaloitus ja paluu alkuvalikkoon.
- ☐ Pakkaa build. Pakkaa build zip-tiedostoksi, jonka nimessä ovat pelin nimi, alusta ja versio, esimerkiksi Pulmapeli-Windows-v1.0.zip.

Valmis kun: Oman pelisi puhtaasta kloonista koottu build käynnistyy ilman editoria, ja zip-tiedosto on nimetty sovitusti.

Tallenna työnäyte: Zip-tiedosto luovutuspakettiin ja build-asetukset omaan päiväkirjaasi.

Työvaihe 3 / 4 · Luovuttakaa paketti asiakkaille

- ☐ Kootkaa paketti. Kootkaa luovutuspaketti avun luovutuslistan mukaan.
- ☐ Tarkistakaa lista. Käykää luovutuslista läpi kohta kohdalta ja rastittakaa vain kohdat, jotka löytyvät paketista.
- ☐ Merkitkää luovutusversio. Merkitkää luovutettu commit versiomerkinnällä eli tagilla v1.0 ja painakaa Push origin.
- ☐ Luovuttakaa sovitulla tavalla. Luovuttakaa paketti asiakkaille sovitulla tavalla, esimerkiksi GitHubin julkaisuna eli releasena tai oppilaitoksen jaetussa kansiossa.
- ☐ Pyytäkää kuittaus. Pyytäkää asiakkailta kuittaus, että paketti on vastaanotettu, ja kirjatkaa se päätöslokiin.

Valmis kun: Luovutuslistan jokainen kohta on paketissa, luovutusversio on merkitty (tag v1.0), ja asiakkaat ovat kuitanneet vastaanoton.

Tallenna työnäyte: Luovutusversion versiomerkintä (tag v1.0) ja päätöslokin kuittausmerkintä repositoryyn.

Työvaihe 4 / 4 · Kirjoita oman työsi kuvaus täsmälinkein

- ☐ Kokoa omat osasi. Käy vastuutaulukko läpi ja listaa jokaisesta pelistä osat, jotka teit.
- ☐ Hae täsmälinkit. Hae jokaiselle osalle commit-linkki ja testitapaus, joka testasi sen.
- ☐ Laske työaika. Laske työaikataulukosta omat tuntisi peleittäin.
- ☐ Kirjoita kuvaus. Kirjoita tiedostoon `docs/tekijat.md` oma osiosi: mitä teit, missä se näkyy ja mitä testasit parisi työstä.
- ☐ Tarkistakaa yhdessä. Lukekaa toistenne osiot ja korjatkaa, jos jokin yhteinen työ puuttuu tai on kahdesti.

Valmis kun: Tekijät-tiedostossa on oma osiosi, jossa jokaisella osalla on commit-linkki ja testitapaus, ja parisi on lukenut sen.

Tallenna työnäyte: `docs/tekijat.md` repositoryyn.

Viikon lopputarkistus: Avatkaa luovutuspaketti puhtaassa kloonissa tai toisella koneella, käynnistäkää jokainen peli ja käykää luovutuslistan kohdat läpi. Jokainen kohta löytyy paketista.

Työnäyte Git-repositoryyn ennen rastia: Luovutusversion versiomerkintä (tag v1.0), luovutuslista, asiakkaiden kuittaus päätöslokissa, oma luovutusbuildisi ja docs/tekijat.md.

Työviikko 18 / 18 – Oman osaamisen esittely

Yhteys kokonaisprojektiin: Pelit on luovutettu. Viimeisellä viikolla ei lisätä uutta, vaan kumpikin tekee oman osaamisensa löydettäväksi ja näyttää sen. Esittelyssä selität omat ratkaisusi ja muutat niitä ohjaajan pyynnöstä.

Viikon tavoite: Kumpikin on näyttänyt omat ratkaisunsa, testinsä ja päätöksensä ja tehnyt muutoksen omaan koodiinsa, ja oma näyttöaineisto on täsmälinkitetty.

Työvaihe 1 / 4 · Kokoa oma näyttömatriisisi täsmälinkein

- ☐ Tarkista perusteet. Tarkista ohjaajalta, minkä tutkinnon perusteiden ja osien mukaan sinut arvioidaan.
- ☐ Luo matriisitiedosto. Kopioi avun pohja tiedostoon `project-docs/omat/OMA-KANSIO/nayttomatriisi.md`.
- ☐ Täytä rivit. Kirjoita jokaiselle omien osiesi vaatimukselle oma tehtäväsi, peli ja täsmälinkki: commit, testitapaus, tiedosto tai tag.
- ☐ Merkitse täydennettävät. Merkitse rivit, joilta työnäyte puuttuu, sarakkeeseen Täydennettävää. Älä keksi työnäytettä.
- ☐ Rastita sivulla. Rastita sivun Näyttömatriisi-näkymässä rivit, joilla on täsmälinkki. Rastit ovat vain sinun muistilistasi.

Valmis kun: Omassa näyttömatriisissasi on jokaisella rivillä täsmälinkki tai merkintä täydennettävästä.

Tallenna työnäyte: `project-docs/omat/OMA-KANSIO/nayttomatriisi.md` repositoryyn.

Työvaihe 2 / 4 · Valmistele oma esittelysi

- ☐ Valitse kaksi ratkaisua. Valitse kaksi omaa ohjelmointiratkaisuasi, jotka osaat selittää parhaiten.
- ☐ Valitse korjattu virhe. Valitse yksi oma virheenkorjausketjusi.
- ☐ Rakenna esittely. Rakenna 8–10 minuutin esittely: peli toiminnassa, kaksi omaa ratkaisua koodissa, korjattu virhe, omat testit ja Git-historia.
- ☐ Kerro tekoälyn käyttö. Jos käytit tekoälyä, kerro miten ja miten tarkistit tuloksen. Ota AI-lokisi mukaan.
- ☐ Harjoittele. Harjoittele esittely kerran parisi kanssa ja pyydä häneltä yksi kysymys, jonka ohjaaja voisi kysyä.

Valmis kun: Esittelyn runko ja valitut täsmälinkit ovat omassa kansiossasi, ja olet harjoitellut esittelyn kerran.

Tallenna työnäyte: Tiedosto `project-docs/omat/OMA-KANSIO/esittely.md` repositoryyn.

Työvaihe 3 / 4 · Esittele ratkaisusi ja tee muutos ohjaajan pyynnöstä

- ☐ Pidä esittely. Pidä valmisteltu esittely ohjaajille.
- ☐ Selitä ratkaisu. Kun ohjaaja valitsee yhden omista ratkaisuistasi, selitä, mitä koodi tekee ja miksi teit sen näin.
- ☐ Tee muutos. Tee ohjaajan pyytämä pieni muutos omaan koodiisi, esimerkiksi muuta kiinteä arvo Inspectorissa säädettäväksi tai lisää uusi ehto.
- ☐ Kokeile muutosta. Kokeile muutosta Play-tilassa ohjaajan nähden ja kerro, miten testaisit sen.
- ☐ Tallenna muutos. Tee muutoksesta commit ohjaajan ohjeen mukaan.

Valmis kun: Olet selittänyt ohjaajan valitseman ratkaisun ja tehnyt siihen toimivan muutoksen Play-tilassa.

Tallenna työnäyte: Muutoksen commit-linkki ja ohjaajan kysymykset omaan päiväkirjaasi.

Työvaihe 4 / 4 · Kirjoita itsearviointi omin esimerkein

- ☐ Kirjoita onnistuminen. Kirjoita yksi asia, joka onnistui, ja liitä siihen täsmälinkki.
- ☐ Kirjoita vaikeus. Kirjoita yksi tilanne, jossa tarvitsit apua, ja miten ratkaisit sen.
- ☐ Arvioi parityötä. Kirjoita, miten sovitte tehtävistä ja testasitte toistenne työtä ja mitä tekisit seuraavassa parityössä toisin.
- ☐ Arvioi oppimista. Kirjoita, mitä osaat nyt Unitystä ja versionhallinnasta sellaista, mitä et vielä osannut työviikolla 1.
- ☐ Kirjoita tekoälyn käytöstä. Jos käytit tekoälyä, kirjoita yksi esimerkki AI-lokista: mitä kysyit, mitä muutit ja miten tarkistit.

Valmis kun: Itsearvioinnissa on viisi kohtaa, jokaisessa on oma esimerkki, ja vähintään kolmessa on täsmälinkki.

Tallenna työnäyte: `project-docs/omat/OMA-KANSIO/itsearviointi.md` ja koko projektipäiväkirja omaan kansioosi repositoryyn.

Viikon lopputarkistus: Ohjaaja valitsee yhden oman ratkaisusi. Selität sen ja teet siihen pienen muutoksen, ja muutos toimii Play-tilassa. Oman näyttömatriisisi jokaisella rivillä on täsmälinkki tai merkintä täydennettävästä.

Työnäyte Git-repositoryyn ennen rastia: project-docs/omat/OMA-KANSIO/nayttomatriisi.md, project-docs/omat/OMA-KANSIO/esittely.md, esittelyssä tehdyn muutoksen commit ja project-docs/omat/OMA-KANSIO/itsearviointi.md.

Viimeiset viisi päivää

Päivä 1 · Oma näyttömatriisi täsmälinkein
Päivä 2 · Oman 8–10 minuutin esittelyn valmistelu
Päivä 3 · Harjoittelu parin kanssa ja muutoksen kokeilu omaan koodiin
Päivä 4 · Puskuri: puuttuvat linkit ja kysymykset ohjaajalle
Päivä 5 · Esittely, muutos ohjaajan pyynnöstä ja itsearviointi

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

prototyyppi	*Pieni pelattava peli.* Peli, joka näyttää yhden peli-idean toiminnassa. ProtoPajan prototyypissä on yksi päämekaniikka, yksi pieni kenttä ja 1–3 minuutin kierros.
päämekaniikka	*Pelin tärkein toiminto.* Toiminto, jota pelaaja toistaa koko kierroksen ajan, esimerkiksi hyppy, laatikon siirto tai hyökkäys.
ydinsilmukka	*Pelin toistuva kierto.* Pelaaja tekee päämekaniikan toiminnon, saa siitä palautetta ja pääsee lähemmäs tavoitetta. Kun silmukka toimii, peli on pelattava.
äänitapahtuma	*Kohta, johon ääni voidaan liittää.* Pelin hetki, esimerkiksi hyppy tai maaliin saapuminen, johon musiikkikurssi voi liittää äänen tai musiikin. Te kuvaatte tapahtuman ja sen paikan koodissa. Ääniä ette tee.
lisätavoite	*Neljäs tai viides peli.* Ajopeli tai tutkimispeli tehdään vain, jos kolme peliä on testattu ja luovutuskunnossa ja lisätyöstä on sovittu. Lisätavoite ei ole työvaihe, eikä se näy etenemisessä.
repository vko 1	*Yhteinen koodivarasto.* Versionhallinnan säilytyspaikka, jossa ovat Unity-projektit, dokumentit ja koko muutoshistoria. Työnäyte on aina repositoryssa, ei tällä sivulla.
commit vko 1	*Tallennettu muutos.* Yksi nimetty muutos versionhallinnassa. Commitin viesti kertoo, mitä muutit, ja tekijätieto näyttää, kumpi teistä sen teki.
push ja pull vko 1	*Lähetys ja haku.* Push lähettää omat commitisi GitHubiin. Pull hakee parisi commitit omalle koneellesi. GitHub Desktopissa painikkeet ovat Push origin ja Pull origin.
LTS vko 1	*Long Term Support, pitkään tuettu versio.* Unityn versio, joka saa korjauksia pitkään. Se sopii projektiin, jonka pitää aueta vielä luovutuksen jälkeen. Molemmilla on oltava täsmälleen sama versio.
scene vko 1	*Unityn näkymä tai taso.* Esimerkiksi alkuvalikko tai pelikenttä. Scene tallentuu yhdeksi tiedostoksi, joten samaa sceneä muokkaa kerrallaan vain yksi.
Play-tila vko 1	*Pelin ajo editorissa.* Unity-editorin tila, jossa peli käynnistyy editorin sisällä. Play-tilassa tehdyt Inspector-muutokset häviävät, kun tila lopetetaan.
tehtävätaulu vko 1	*Yhteinen työlista.* Taulu, esimerkiksi Trello, jossa jokainen työtehtävä on kortti: tavoite, vastuuhenkilö, valmis kun -ehto ja linkki työnäytteeseen. Taulusta näkee, kuka tekee mitä.
.gitignore vko 1	*Pois jätettävät tiedostot.* Tiedosto, joka kertoo Gitille, mitä ei tallenneta. Unity-projektista jätetään pois esimerkiksi kansiot Library, Temp ja Builds, koska Unity luo ne itse.
pienin toimiva versio vko 2	*Pelin ensimmäinen valmis muoto.* Versio, jossa päämekaniikka, tavoite ja kierroksen loppu toimivat. Se tehdään ensin. Kaikki muu on lisää.
ristiriita vko 3	*Merge conflict.* Kaksi ihmistä on muuttanut saman tiedoston samaa kohtaa, eikä Git osaa valita, kumpi muutos jää. Unityn scene-tiedoston ristiriita on työläs ratkaista, ja siksi samaa sceneä muokkaa kerrallaan yksi.
GameObject vko 3	*Scenen esine.* Mikä tahansa sceneen lisätty asia: pelaaja, seinä, kamera tai painike. GameObjectin toiminta tulee siihen liitetyistä komponenteista.
komponentti vko 3	*GameObjectin ominaisuus.* GameObjectiin liitetty osa. Esimerkiksi Rigidbody 2D lisää painovoiman, ja oma C#-skriptisi lisää pelin säännön. Komponentteja muokataan Inspector-ikkunassa. Unity on ohjelmistokomponenttikirjasto, koska peli kootaan sen valmiista ja omista komponenteista.
C#-skripti vko 3	*Oma koodi Unityssä.* Tiedosto, johon kirjoitatte pelin toiminnan C#-kielellä. Skripti liitetään GameObjectiin komponenttina.
build vko 3	*Koottu peliversio.* Versio, jonka Unity kokoaa projektista ajettavaksi ilman editoria, esimerkiksi Windows-ohjelmaksi tai selaimessa toimivaksi peliksi. Build testataan aina erikseen, koska se voi toimia eri tavalla kuin editori.
collider vko 4	*Törmäysmuoto.* Komponentti, joka antaa GameObjectille muodon törmäyksiä varten. Trigger-collider ei pysäytä liikettä, vaan ilmoittaa skriptille, kun jokin osuu siihen, esimerkiksi maaliin.

T01 vko 4	*Testitapauksen tunnus.* T tarkoittaa testitapausta, ja numero yksilöi sen: T01 on ensimmäinen testitapaus, T02 toinen. Odotettu tulos kirjataan aina ennen ajoa, ja sama tunnus seuraa testiä testimatriisiin.
UI vko 5	*User interface, käyttöliittymä.* Pelin painikkeet, tekstit ja valikot. Unityn GameObject-valikon UI-kohdasta lisätään esimerkiksi painike ja teksti.
asset vko 5	*Valmis aineisto.* Peliin tuotava kuva, sprite eli 2D-pelin kuva, fontti tai muu valmis aineisto. Valmiita assetteja saa käyttää niiden käyttöehtojen mukaan, ja lähde kirjataan assettiluetteloon.
prefab vko 5	*Uudelleenkäytettävä GameObject.* Tallennettu valmis GameObject, jota voi käyttää monessa scenessä. Kun muokkaat prefabia, muutos näkyy kaikissa sen kopioissa. Esimerkiksi alkuvalikon voi tehdä prefabiksi ja käyttää kaikissa peleissä.
tag vko 6	*Versiomerkintä.* Nimi, joka kiinnitetään yhteen commitiin, jotta juuri se versio löytyy myöhemmin. Asiakaskokeilun versio on v0.1 ja luovutusversio v1.0.
JSON vko 7	*Tiedon tekstimuoto.* Tekstimuoto, jolla tietoa tallennetaan tiedostoon nimettyinä kenttinä. Pelin kenttä- tai asetusdata voi olla JSON-tiedostossa, jonka peli lukee käynnistyessään.
ScriptableObject vko 7	*Unityn datatiedosto.* Unityn tapa tallentaa dataa omaksi tiedostokseen projektin sisälle, esimerkiksi kentän asetukset. Dataa muokataan Inspectorissa ilman koodia.
API vko 7	*Ohjelmointirajapinta.* Valmiiden toimintojen joukko, jota oma koodi kutsuu. Unityn API:sta käytätte esimerkiksi fysiikkaa (Physics2D), scenen vaihtoa (SceneManager) ja aikaa (Time).
vertaisarvio vko 9	*Parin työn arviointi.* Testaat parisi toteutuksen sovittuja vaatimuksia vasten ja kirjaat havainnot. Vertaisarvio koskee toteutusta, ei ihmistä.
virheenkorjausketju vko 9	*Virheen koko kulku.* Havainto → toistamisohje → syy → korjauscommit → uusintatesti → regressiotesti. Projektissa tehdään vähintään neljä ketjua, ja kumpikin tekee vähintään kaksi oman koodinsa virheestä.
regressiotesti vko 9	*Ettei korjaus rikkonut muuta.* Korjauksen jälkeen ajettava testi, jolla varmistat, että muu peli toimii yhä. Yleensä se on pelin peruskierros alusta loppuun.
vastaanottotesti vko 16	*Ohjeen testi ulkopuolisella.* Projektin ulkopuolinen henkilö avaa Unity-projektit ja käynnistää pelit pelkän kirjallisen ohjeen avulla. Jokainen kohta, jossa hän epäröi, korjataan ohjeeseen.

Näyttömatriisi – 32 osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelvata useaan kohtaan.

Ohjelmointi

□ **käyttää ohjelmointieditoria tai kehitysympäristöä** – Oma työnäyte: Oma Unity-asennus ja versiotiedot, oma muutos kokeiluprojektiin ja oma skripti kokeilukenttään (työviikot 1, 3).

□ **etsii ja korjaa virheitä ohjelmakoodista** – Oma työnäyte: Kummankin vähintään kaksi virheenkorjausketjua oman koodin virheestä: havainto, toistamisohje, syy, korjauscommit, uusintatesti ja regressiotesti (työviikot 9, 13, 16).

□ **testaa ohjelman toimintoja** – Oma työnäyte: Testimatriisin testitapaukset, joissa opiskelija on testin tekijä: odotettu tulos ennen ajoa, parin osien testaus ja koottujen versioiden testit (työviikot 4, 8, 14).

□ **käyttää rakenteista ohjelmointia toteutuksissa** – Oma työnäyte: Omat skriptit päämekaniikkaan ja kierroksen logiikkaan: metodit, ehdot, silmukat ja kentät, esimerkiksi kentän luku datasta silmukoilla (työviikot 4, 7, 10).

□ **kirjoittaa ylläpidettävää ohjelmakoodia** – Oma työnäyte: Valikon uudelleenkäyttö prefabina, toiston poisto omaan metodiin ja oman koodin siistiminen nimineen ja alkukommentteineen (työviikot 8, 15).

□ **tulkitsee suunnitelmia ja toteuttaa käyttöliittymän tai sen osia** – Oma työnäyte: Oma käyttöliittymäosa vastuutaulukon mukaan: alkuvalikko ja ohje tai loppuruutu, uudelleenaloitus ja palaute suunnitelman mukaan (työviikot 5, 8, 11).

□ **tulkitsee suunnitelmia ja toteuttaa ohjelmiston toimintoja** – Oma työnäyte: Oma osa päämekaniikasta ja kierroksesta suunnitelman ja kortin odotetun tuloksen mukaan (työviikot 4, 7, 10).

□ **sopii tehtävistä tiimin muiden jäsenten kanssa** – Oma työnäyte: Vastuutaulukko, tehtävätaulun kortit vastuuhenkilöineen ja jokaisen toteutusviikon kahden osan sopiminen (työviikot 2, 4, 7).

□ **etsii ratkaisuvaihtoehtoja ja ratkoo ongelmia yhdessä tiimin kanssa** – Oma työnäyte: Projektijaon vertailu omin mittauksin ja ristiriitaharjoitus, jossa kumpikin ratkaisee ristiriidan (työviikot 2, 3).

□ **arvioi ratkaisujen toimivuuden yhdessä tiimin kanssa** – Oma työnäyte: Vertaisarvio parin osista ja toistensa pelien testaus koottuina versioina (työviikot 9, 14).

□ **arvioi omaa toimintaa tiimin jäsenenä** – Oma työnäyte: Aukkolista ja kolmen virkkeen arvio omasta toiminnasta työviikolla 12 sekä itsearviointi täsmälinkein työviikolla 18 (työviikot 12, 18).

Ohjelmistokehittäjänä toimiminen

□ **selvittää kehitystiimin kanssa asiakkaan tarpeet** – Oma työnäyte: Kysymyslista asiakkaille ja ohjaajille vastauksineen ja aiheiden vahvistuspyyntö asiakkaille (työviikot 1, 2).

□ **viestii tekniset asiat asiakaslähtöisesti** – Oma työnäyte: Esittely asiakkaille ilman teknistä sanastoa (roolit vaihtuvat työviikkojen 6 ja 12 välillä) ja oma osuus jatkotyöstöohjeesta (työviikot 6, 12, 15).

□ **osallistuu version katselmointiin** – Oma työnäyte: Asiakaskokeilu ja asiakaskatselmointi: toinen esittelee ja toinen kirjaa palautteen asiakkaan sanoin, roolit vaihtuvat (työviikot 6, 12).

□ **asettaa kehitystiimin kanssa toteutettavat toiminnot tärkeysjärjestykseen** – Oma työnäyte: Pienimmän toimivan version rajaus, palautekorttien järjestys ja korjausten järjestys asiakkaiden kanssa (työviikot 2, 6, 12).

□ **jakaa kehitystiimin kanssa toteutettavat toiminnot tehtäviksi** – Oma työnäyte: Ensimmäisen pelin toiminnot tehtävätaulun korteiksi valmis kun -ehtoineen (työviikko 2).

□ **suunnittelee ja arvioi kehitystiimin kanssa tehtävien toteuttamista** – Oma työnäyte: Tuntiarviot korteissa ja oma arvio–toteumavertailu työaikataulukosta (työviikot 2, 6).

□ **kehittää ohjelmiston toimintalogiikkaa** – Oma työnäyte: Kierroksen päättymisen logiikka, ratkaisun tunnistus tai ajastin voitto- ja häviöehtoineen, loppu vain kerran (työviikot 4, 11).

□ **valitsee ohjelmistoon sopivan tietovaraston** – Oma työnäyte: Sama kenttä kahdella tallennustavalla (ScriptableObject ja tekstitiedosto tai JSON), muutoskokeilu ja perusteltu valinta päätöslokissa (työviikko 7).

□ **toteuttaa yhteyden tietovarastoon** – Oma työnäyte: Kentän luku datasta, alkutilanteeseen palautus samasta datasta tai kolmannen pelin asetukset datasta; kumpikin tekee vähintään yhden (työviikot 7, 8, 10).

□ **hyödyntää rajapintoja ja käsittelee tietoa** – Oma työnäyte: Unityn API:n käyttö (esimerkiksi Physics2D, SceneManager, JsonUtility) ja datan käsittely virheellisen syötteen kestävästi (työviikot 7, 10).

□ **arvioi ohjelmiston tietoturvaa** – Oma työnäyte: Oma tietoturva-arvio: luovutettavien tiedostojen tarkistus, ulkoisten pakettien alkuperä ja testitapaus virheellisellä datalla (työviikko 14).

□ **käyttää versionhallintaa** – Oma työnäyte: Kummankin commitit koko projektin ajan, versiomerkinnät v0.1, v0.2, v0.9 ja v1.0 (työviikot 1, 17).

□ **liittää ohjelman osan olemassa olevaan versioon** – Oma työnäyte: Oma yhdistämiscommit ristiriitaharjoituksesta ja oman prefabin tai scene-osan liittäminen parin muutosten päälle yhteiseen versioon (työviikot 3, 5).

□ **julkaisee ohjelman tuotantoympäristöön** – Oma työnäyte: Oman pelin luovutusbuild puhtaasta kloonista, nimetty paketti ja luovutus asiakkaille sovitulla tavalla (työviikko 17).

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

□ **ottaa käyttöön ja konfiguroi ohjelmistokomponenttikirjaston käyttöön soveltuvan kehittämisympäristön** – Oma työnäyte: Unity Hub, sovittu versio ja moduulit, projektin luonti ja versionhallinta-asetukset (Force Text, Visible Meta Files) (työviikot 1, 3).

□ **selvittää ohjelmistokomponenttikirjaston tarjoamat mahdollisuudet ja rajoitteet** – Oma työnäyte: Kolme omaa havaintoa Unityn mahdollisuuksista ja rajoitteista commit-linkkeineen tai kuvakaappauksineen (työviikko 6).

□ **käyttää ohjelmistokomponenttikirjaston tärkeimpiä toimintoja ja työkaluja** – Oma työnäyte: Komponentit, colliderit ja triggerit, Rigidbody 2D, Canvas ja painikkeet, prefabit ja Build Profiles omissa osissa (työviikot 4, 7, 10).

□ **tuo kehittämisympäristöön ulkoisia komponentteja** – Oma työnäyte: Grafiikkapaketin tuonti ja käyttöehtojen tarkistus vastuutaulukon mukaan (grafiikkavastaava vaihtuu), Input System -paketti (työviikot 5, 7, 10).

□ **suunnittelee, toteuttaa ja testaa ohjelmiston ohjelmistokomponenttikirjastoa käyttäen** – Oma työnäyte: Pelien suunnittelu, toteutus omina osina ja testaus koottuina versioina komponenttikirjastoa käyttäen (työviikot 2, 8, 11, 14).

□ **julkaisee ohjelmiston asiakkaan ympäristöön** – Oma työnäyte: Oman pelin build asiakkaan alustalle puhtaasta kloonista ja sen testaus ilman editoria (työviikko 17).

□ **dokumentoi ohjelmiston sovitulla tavalla** – Oma työnäyte: Oma osuus avaamis- ja jatkotyöstöohjeesta, kokeiltu parilla ja työviikolla 16 ulkopuolisella (työviikko 15).

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.